

POLYTECHNIQUE MONTRÉAL

affiliée à l'Université de Montréal

**Génération de trajectoire robotique sans collision dans un environnement
dynamique utilisant des modèles de Flow Matching**

FRÉDÉRICK LANTHIER

Département de génie mécanique

Mémoire présenté en vue de l'obtention du diplôme de *Maîtrise ès sciences appliquées*
Génie mécanique

Septembre 2026

POLYTECHNIQUE MONTRÉAL

affiliée à l'Université de Montréal

Ce mémoire intitulé :

**Génération de trajectoire robotique sans collision dans un environnement
dynamique utilisant des modèles de Flow Matching**

présenté par **Frédéric LANTHIER**

en vue de l'obtention du diplôme de *Maîtrise ès sciences appliquées*

a été dûment accepté par le jury d'examen constitué de :

Danielle DUBOIS, présidente

Sofiane ACHICHE, membre et directeur de recherche

Jérôme LE NY, membre et codirectrice de recherche

Jean TREMBLAY, membre

Joseph BROWN, membre externe

DÉDICACE

*À ma blonde que j'aime, mes parents,
et mes amis des labs. . .*

REMERCIEMENTS

Texte / Text.

RÉSUMÉ

Le résumé est un bref exposé du sujet traité, des objectifs visés, des hypothèses émises, des méthodes expérimentales utilisées et de l'analyse des résultats obtenus. On y présente également les principales conclusions de la recherche ainsi que ses applications éventuelles. En général, un résumé ne dépasse pas trois pages.

Le résumé doit donner une idée exacte du contenu du mémoire ou de la thèse. Ce ne peut pas être une simple énumération des parties du manuscrit. Le but est de présenter de façon précise et concise la nature, l'envergure de la recherche, les sujets traités, les questions de recherche ou les hypothèses soulevées, les méthodes utilisées, les principaux résultats ainsi que les conclusions retenues. Un résumé ne doit jamais comporter de références ou de figures.

ABSTRACT

Written in English, the abstract is a brief summary similar to the previous section (Résumé). However, this section is not a word for word translation of the abstract in French.

The abstract is a brief statement of the subject matter, objectives, research questions or hypotheses, experimental methods and analysis of results. It also presents the main research conclusions and their possible applications. In general, an abstract should not exceed three pages.

The abstract should provide an exact idea of the thesis or dissertation's contents and it cannot be a simple enumeration of the manuscript's parts. The goal is to precisely and concisely present the nature and scope of the research. An abstract should never include references or figures. If the thesis or the dissertation is in English, the résumé (French-language abstract) should come first followed by the abstract.

TABLE DES MATIÈRES

LISTE DES TABLEAUX

LISTE DES FIGURES

LISTE DES SIGLES ET ABRÉVIATIONS

API	Application Programming Interface
ARA	Assistive Robotic Arm
BC	Behavior Cloning
CBF	Constraint Barrier Function
CFM	Constraint Flow Matching
CPU	Central Processing Unit
DDL	Degré de liberté
FM	Flow Matching
GPU	Graphics Processing Unit
IA	Intelligence Artificielle
JSON	JavaScript Object Notation
LLM	Large Language Model
QP	Quadratic Problem
RAF	Robotic Assisted Feeding
RDF	Robot Distance Field
RGB-D	Red-Green-Blue + Depth
RL	Reinforcement Learning
RLHF	Reinforcement Learning from Human Feedback
ROS	Robot Operating System
SAM	Segment Anything Model
SDF	Signed Distance Field
SO	Specific objective
VLM	Vision Language Model
YAML	YAML Ain't Markup Language

LISTE DES ANNEXES

CHAPITRE 1 INTRODUCTION

Présenter la problématique de recherche, comme quoi l'utilisation des modèles d'IA générative pour des tâches robotiques réelles ne sont pas couramment utiliser puisqu'ils nécessite le positionnement d'une caméra externe, parce que c'est dans des environnements dynamiques et que si on veut qu'ils soient réactifs aux collisions, il faut entraîner le modèle sur plusieurs centaines de données au lieu d'une quarantaine pour une tâche afin de lui faire faire apprendre l'évitement de collision. cela complexifie alors parler aussi des robots collaboratifs qui permettent de réaliser des tâches, mais que ces tâches sont pour la plupart hardcoder et ne réagissent pas aux stimuli environnants, ce qui peut générer des trajectoires dangereuses.

Parler de la motivation de la recherche. Pour moi, serait que la génération de trajectoire par intelligence artificielle, pour ne pas accumuler des centaines de données pour faire de l'évitement de collision doit avoir un système d'évitement de collision sur la trajectoire réalisée. Ce système peut se trouver lors de l'inférence du modèle ou bien après l'inférence du modèle. Lors de l'inférence du modèle, c'est trop long pour des environnements dynamiques et après il faut considérer l'environnement, ce qui nécessite l'entraînement d'un réseau que pour l'environnement. De plus, les systèmes actuels nécessitent une caméra externe calibrée adéquatement, ce qui n'est pas utilisable pour un usage quotidien. Ce qu'on propose c'est un système d'évitement de collision pour des environnements dynamiques pour des environnements non vues pendant l'entraînement avec une seule caméra au niveau de l'effecteur du robot.

Faire un paragraphe par rapport à la structure du mémoire. Celle de Félix c'était : Chapter 2 reviews the literature on robot-assisted feeding, highlighting current challenges in perception, task planning, and manipulation. Chapter 3 defines the specific scope and research objectives of this project. Chapter 4 details the design of the system, describing the system architecture, the integration of Vision-Language Models (VLMs) for autonomous food acquisition planning, and the implementation of the food perception and manipulation modules. Chapter 5 presents the experimental results, including an analysis of VLM-based food acquisition planning performance and a validation of the system on a physical robot. Finally, Chapter 6 summarizes the main contributions, discusses the system's limitations, and proposes directions for future research.

Donc nous cela pourrait être "Le chapitre 2 présente la revue de littérature en lien avec la génération de trajectoires robotiques en utilisant l'IA générative mettant en évidence l'exécution des trajectoires la vision par ordinateur et le suivi de cible ainsi que l'évitement de

collision. Le chapitre 3 présente la question de recherche ainsi que les sous objectifs associées pour répondre à cette hypothèse. Le chapitre 4 présente le design du système, mettant de l'avant son architecture, l'intégration des systèmes de vision par ordinateur pour détecter et suivre la cible, l'acquisition des données pour entraîner le modèle de Flow Matching, la génération des trajectoires générées par IA ainsi, la capture des obstacles, le filtre de sécurité utilisé et l'intégration du temps réel. Le chapitre 5 présente les résultats expérimentaux, la validation du modèle géométrique de sécurité, la qualité de la trajectoire nominale, l'analyse des vitesses, la sécurité et l'évitement d'obstacle, les performances computationnelles ainsi que les limites du système. "

CHAPITRE 2 REVUE DE LITTÉRATURE

2.1 Génération de trajectoires robotisées

Objectif de la section : Analyser le développement théorique et les limites connues des méthodes de génération de mouvements afin de situer le choix du *Flow Matching* conditionnel comme planificateur d'intention nominale.

2.1.1 Planification classique par échantillonnage et optimisation

Cette sous-section présente les approches algorithmiques traditionnelles de la planification de mouvements. Les méthodes basées sur l'échantillonnage de l'espace des configurations, telles que RRT* [?] implémentées dans l'environnement OMPL [?], offrent une complétude probabiliste mais tendent à générer des trajectoires géométriques rugueuses, dont la sélection optimale dépend fortement de la requête et de la scène [?]. À l'inverse, les méthodes fondées sur l'optimisation locale (CHOMP [?], TrajOpt [?]) lissent la trajectoire en minimisant des fonctionnels de coût cinématiques sous contraintes d'obstacles, typiquement formulées sur un champ de distance signé de la scène [?]. Elles manifestent toutefois une sensibilité prononcée au profil d'initialisation, risquant la convergence vers des minima locaux sous-optimaux, parfois incompatibles avec les contraintes de collision selon l'initialisation et la qualité du champ de distance. Cette dépendance à l'initialisation motive d'ailleurs les approches récentes d'amorçage (*warm-starting*) par modèles génératifs [?, ?].

2.1.2 Apprentissage par démonstration, Behavior Cloning et RL

Cette partie évalue les lois de contrôle fondées sur l'apprentissage supervisé et par renforcement, dont les fondements ont été formalisés très tôt dans la littérature sur l'apprentissage par démonstration [?]. Le clonage de comportement (*Behavior Cloning* - BC) standard utilise l'apprentissage profond supervisé pour régresser des actions à partir de données expertes [?], comme l'illustrent les architectures d'acquisition alimentaire fondées sur l'imitation visuelle [?, ?], mais demeure assujetti à l'erreur cumulative (*compounding errors*) et au risque d'effondrement de la multimodalité sous l'usage de fonctions de perte de type Erreur Quadratique Moyenne (MSE). Ce constat a motivé les formulations implicites ou fondées sur l'énergie, qui modélisent des distributions d'actions multivaluées et discontinues plutôt qu'une régression directe [?, ?], ainsi que l'usage de représentations apprises de façon auto-supervisée pour améliorer la généralisation [?]. L'Apprentissage par Renforcement (*Reinforcement Lear-*

ning - RL) résout la multimodalité par exploration active [?, ?], mais présente une inefficacité de collecte de données (*sample inefficiency*) ainsi que des risques de collision délicats à gérer lors de l'apprentissage direct sur plateforme physique. Une alternative consiste à encoder le mouvement comme un système dynamique stable garantissant la convergence vers la cible [?].

2.1.3 Modèles génératifs, politiques de diffusion et Flow Matching

Cette section étudie la fine pointe des planificateurs génératifs continus, dont plusieurs revues récentes dressent le panorama [?, ?]. Les architectures basées sur la *Diffusion Policy* [?], héritières des modèles probabilistes de débruitage [?] et de la planification par diffusion [?, ?], modélisent efficacement les distributions complexes et multimodales de trajectoires expertes par débruitage stochastique, y compris à partir d'observations tridimensionnelles [?], mais leur latence de calcul limite parfois leur fréquence d'inférence en boucle fermée. Cette latence a suscité une famille de méthodes d'accélération par distillation ou contrainte de cohérence réduisant le nombre d'étapes d'inférence [?, ?, ?]. Le *Conditional Flow Matching* (CFM) [?, ?, ?], dont l'application à l'apprentissage de politiques de manipulation a été démontrée à partir de nuages de points [?], et ses formulations accélérées (*Consistency Flow* [?], *Meanflow Policy* [?, ?], flux en continu [?]) s'affranchissent de cette lourdeur en s'appuyant sur des équations différentielles ordinaires (ODE) déterministes et redressées, autorisant une génération de trajectoires nominales de haute qualité en un nombre réduit d'étapes d'inférence (NFE). Ces formulations ont été déclinées pour la planification de mouvement de manipulateurs [?, ?] et étendues aux variétés riemanniennes afin de respecter la géométrie de l'espace des poses [?].

2.2 Perception visuelle et mémoire spatiale

Objectif de la section : Documenter les mécanismes permettant de convertir un flux de données visuelles bidimensionnelles et bruitées en un signal de conditionnement géométrique stable et persistant.

2.2.1 Vision-langage, segmentation zéro-shot et suivi visuel

Ce segment traite de l'extraction sémantique et de l'ancrage de la tâche. Les architectures *Vision-Language-Action* (VLA) monolithiques [?, ?] unifient la perception et l'action — un paradigme dont plusieurs revues retracent l'évolution [?, ?] — mais présentent une latence d'inférence parfois incompatible avec le contrôle à haute fréquence [?]. Les approches modulaires permettent de découpler l'exécution : un modèle de vision-langage interprète l'invite textuelle de la tâche et ancre l'instruction dans la scène [?, ?, ?, ?], tandis que le modèle

Segment Anything 3 (SAM3) [?], héritier du modèle fondateur de segmentation promptable SAM [?], segmente l’instance correspondant précisément à cette invite textuelle. Le modèle SAM2 [?] assure ensuite la propagation temporelle et le suivi du masque sémantique sur le flux de trames visuelles.

2.2.2 Reconstruction RGB-D et représentation par nuages de points

Cette sous-section analyse le passage de l’espace image bidimensionnel vers l’espace cartésien tridimensionnel à l’aide de capteurs RGB-D. La rétroprojection géométrique des masques sémantiques isolés par SAM permet d’extraire la géométrie tridimensionnelle de la cible et des obstacles sous forme de nuages de points. Le conditionnement des politiques génératives directement sur des nuages de points s’est imposé comme une alternative robuste aux observations purement image, car il capture explicitement la structure tridimensionnelle de la scène [?, ?, ?, ?], y compris à partir d’une vue unique malgré les occlusions induites par l’effecteur [?] ou en suivant la pose d’objets clés plutôt que le nuage brut [?]. Certains travaux montrent toutefois qu’une fusion 2D enrichie de pseudo-3D peut approcher ces performances à moindre coût d’acquisition [?, ?]. Cette technique capture l’organisation spatiale locale et non structurée de l’environnement sans nécessiter de modèles CAO ou de primitives analytiques connus a priori.

2.2.3 Cartes persistantes face aux occlusions et pertes de détection

Cette section étudie les couches de mémoire spatio-temporelle explicites. En environnement non structuré et dynamique, les occlusions physiques causées par le corps du robot lui-même ou par les modifications de la scène perturbent l’inférence des politiques visuo-motrices [?]. La cartographie d’environnements dynamiques, qui sépare les structures statiques des entités mobiles, constitue un cadre établi pour ce problème [?], de même que l’estimation de pose tenant compte de l’incertitude sous observation partielle [?]. L’état de l’art documente l’intégration de structures de données cartographiques persistantes (grilles d’occupation historiques, voxelisation locale) pour conserver la trace spatiale de la cible et des obstacles durant les pertes de détection ou les masquages temporaires.

2.3 Représentations géométriques pour la collision

Objectif de la section : Évaluer les méthodes de modélisation de l’encombrement spatial du robot et des obstacles pour formuler des métriques de distance différentiables.

2.3.1 ESDF de l’environnement et limites computationnelles

Cette section se concentre sur les champs de distance signés euclidiens (*Euclidean Signed Distance Fields* - ESDF) centrés sur la scène. Des bibliothèques de cartographie volumétrique telles que OctoMap [?], Voxblox [?] ou nvblox [?] maintiennent un champ de distance continu de l’espace libre pour l’évaluation immédiate des collisions, et des transformées de distance calculées sur GPU permettent un évitement réactif à partir d’un nuage de points en direct [?]. Cependant, leur mise à jour peut devenir coûteuse lorsque la scène est dynamique, dense ou fortement bruitée, surtout si l’on cherche à maintenir une boucle de correction à haute fréquence.

2.3.2 Primitives conservatives et modèles géométriques simplifiés

Cette partie examine la simplification géométrique requise pour le calcul en temps réel. L’approche classique en ingénierie consiste à encapsuler les liens cinématiques du robot sous forme de volumes englobants discrets (enveloppes de sphères ou de capsules), massivement parallélisés sur GPU (comme développé dans cuRobo [?]). Ces représentations conservatives autorisent des requêtes de collision rapides, et des travaux récents les rendent même conscientes de l’outil monté à l’effecteur pour ajuster la marge en temps réel [?], mais introduisent un caractère restrictif qui réduit l’espace libre exploitable dans les zones confinées.

2.3.3 SDF du robot, RDF et approximation par polynômes de Bernstein

Cette sous-section présente le changement de paradigme des *Robot Distance Fields* (RDF) [?]. Au lieu de cartographier la scène, la géométrie du bras articulé est modélisée par une approximation polynomiale de Bernstein. Cette formulation fournit une approximation différentiable du SDF du robot, rendant ses gradients analytiques directement interrogeables par rapport aux points de l’environnement, ce qui a récemment été exploité pour l’évitement réactif de collisions dans un cadre de commande à priorité de tâches [?]. D’autres approches apprennent plutôt un SDF composite du robot par réseaux de neurones, couplé à de l’optimisation stochastique [?]. On note cependant que l’usage d’opérateurs de pooling (*min* sur les liens et *soft-min* sur les points) implique que la fonction complète n’est pas strictement continue-dérivable (C^∞) partout.

2.4 Sécurité réactive et Control Barrier Functions

Objectif de la section : Analyser les lois de commande réactives issues de la théorie du contrôle afin d'imposer une contrainte de sécurité en vitesse sous certaines hypothèses.

2.4.1 Champs de potentiel et méthodes réactives classiques

Cette partie traite des lois de répulsion classiques. Les approches par Champs de Potentiel Artificiels (APF) [?] génèrent des forces répulsives virtuelles à proximité des obstacles pour dévier localement le robot. Bien que légères computationnellement, ces méthodes présentent des limites structurelles bien connues, notamment des oscillations prononcées près des parois, l'absence de garanties rigoureuses face à des dynamiques complexes et des risques majeurs de blocage au sein de minima locaux concaves. Le concept demeure néanmoins exploité dans les approches génératives récentes, où un champ de potentiel appris guide l'imitation vers des mouvements sûrs [?].

2.4.2 CBF, invariance avant et projection de commande

Cette section détaille les fondements des fonctions barrières de contrôle (*Control Barrier Functions* - CBF) [?, ?]. Pour un ensemble sûr caractérisé par une fonction barrière locale $h(q)$, la contrainte différentielle $\nabla h(q)^T \dot{q} \geq -\kappa h(q)$ permet de formuler des conditions de sécurité vérifiables localement, et garantit l'invariance avant de l'ensemble sûr. Lorsque cette contrainte est formulée en vitesse articulaire, le filtre agit comme une projection de type QP-CBF [?], équivalent à une projection minimale de la vitesse nominale pour respecter la frontière d'évitement. L'extension aux systèmes mécaniques de degré relatif deux, où la commande s'exerce au niveau des efforts, a également été formalisée [?].

2.4.3 Limites des CBF sous perception bruitée et contrôle discret

Cette sous-section analyse les contraintes liées à l'implémentation physique réelle du contrôle basé sur les CBF. Contrairement aux formulations théoriques continues, l'application des barrières sur une plateforme réelle souffre du bruit de mesure des nuages de points et des délais d'échantillonnage du contrôle discret, ce qui motive les formulations en temps discret de la condition barrière [?]. S'ajoute le problème de compatibilité des contraintes, lorsque la barrière exige un effort de commande excédant les bornes physiques de l'actionneur [?]. De plus, les non-linéarités induites par les approximations géométriques lisses du robot et le suivi par un contrôleur en position bas niveau limitent la validité absolue des preuves d'invariance globale.

2.5 Couplage entre modèles génératifs et filtres de sécurité

Objectif de la section : Examiner les différentes stratégies d'unification de l'IA générative et des lois de commande sécuritaires afin de positionner la méthode de découplage retenue.

2.5.1 Contraintes intégrées dans la génération

Cette section analyse les cadres intégrant la sécurité pendant le processus de génération de la trajectoire (tels que *SafeFlowMatcher* [?], *SafeFlow* [?], CASF [?] ou *OmniGuide* [?]). Ces méthodes appliquent des contraintes souples, des métriques riemanniennes ou des corrections itératives à chaque étape de l'intégration de l'ODE générative (guidage intra-ODE). Plusieurs variantes imposent ces contraintes via des fonctions barrières et de Lyapunov dans le processus de diffusion [?], via une formulation unifiée admettant égalités et inégalités [?], ou en garantissant l'admissibilité dynamique des trajectoires planifiées [?]. D'autres conditionnent la génération sur la structure de la scène pour amorcer un planificateur sous contraintes [?, ?]. Bien qu'efficaces en simulation, elles augmentent le temps de calcul et induisent un risque de dérive distributionnelle (*distributional drift*) en forçant le modèle dans des états latents hors distribution.

2.5.2 Filtres de sécurité appliqués à l'exécution

Cette partie étudie les approches appliquant un mécanisme de filtrage sur la trajectoire finale produite par l'IA (frameworks inspirés de PACS [?]). Ces méthodes préservent la distribution nominale apprise en boucle ouverte en ne modifiant la trajectoire qu'au moment du déploiement ou de l'exécution physique, agissant comme un bouclier cinématique externe sans perturber la dynamique interne du modèle génératif. D'autres travaux corrigent localement la politique à l'exécution à partir de signaux épars [?], compensent explicitement le délai d'inférence entre l'observation et l'exécution [?], ou activent une replanification à haute fréquence sans réentraînement [?].

2.5.3 Découplage entre intention nominale et sécurité réactive

Cette sous-section apporte la justification de l'architecture découplée retenue. L'analyse des sections précédentes motive l'isolation fonctionnelle complète entre un planificateur *Flow Matching*, qui gère l'intention globale à long terme sans risque de dérive distributionnelle, et un filtre de sécurité réactif local agissant en continu à l'exécution [?, ?]. Ce découplage permet de concilier l'expressivité des distributions apprises et l'impératif de réactivité requis pour intercepter les risques de collision sur la plateforme réelle.

2.6 Synthèse critique et positionnement

Objectif de la section : Résumer les lacunes de l'état de l'art, formaliser l'hypothèse de recherche et présenter un tableau comparatif synthétique des architectures de la littérature.

2.6.1 Lacunes identifiées dans la littérature

L'analyse croisée de la littérature scientifique contemporaine met en exergue trois lacunes majeures :

1. **L'hypothèse d'une perception parfaite :** Les frameworks actuels validant l'alliance du *Flow Matching* et des CBF supposent des obstacles analytiques parfaits connus a priori, se montrant inadaptés face aux nuages de points 3D bruts et bruités.
2. **L'absence de persistance temporelle des modèles génératifs :** Les planificateurs génératifs souffrent d'une vulnérabilité cinématique lors d'occlusions temporaires de la cible, faute d'une couche cartographique persistante capable de stabiliser les points de conditionnement.
3. **Le coût numérique des représentations globales :** Les approches basées sur la mise à jour de champs de distance globaux ou de *Neural SDF* s'avèrent trop lourdes pour garantir une boucle de rétroaction à haute fréquence face à des obstacles imprévus.

2.6.2 Positionnement de l'architecture proposée

Pour répondre à ces limites, cette thèse soutient la pertinence d'une *architecture découplée entre génération nominale et filtrage réactif de sécurité*. L'intention globale est générée en boucle ouverte par un modèle *Flow Matching* conditionné par une observation géométrique persistante issue de la perception RGB-D et du suivi SAM2, tandis que la sécurité immédiate est déléguée à un filtre de projection CBF articulaire bas niveau s'appuyant sur une approximation polynomiale de Bernstein du SDF local du robot.

2.6.3 Hypothèse de recherche

L'hypothèse centrale de cette recherche s'énonce ainsi :

« L'utilisation d'un filtre SDF-CBF fondé sur un SDF local du robot, approximé par des polynômes de Bernstein et alimenté par une carte perceptuelle persistante, permet de corriger en temps réel les vitesses nominales générées par un modèle *Flow Matching*, afin d'améliorer l'exécution sécuritaire de trajectoires dans des

environnements non structurés et inconnus, sans réentraîner le modèle génératif ni construire un champ de distance global de l’environnement. »

Le Tableau ?? résume le positionnement de l’approche proposée vis-à-vis des frameworks de référence de la littérature.

Tableau 2.1 Tableau comparatif des frameworks de planification et de sécurité de l’état de l’art.

Cadre / Méthode	Modèle nominal	Sécurité	Représentation	Correction	Occlusions	Réactivité
Diffusion Policy	Diffusion	Implicite	Espace latent	Aucune / boucle ouverte	Non	Limitée par le débrui- tage
OmniGuide	Flow Mat- ching	Souple	Grille / éner- gie	Guidage intra-ODE	Non	Moyenne
CASF	Streaming Flow	Souple	Neural SDF	Déformation intra-ODE	Partielle	Élevée mais coûteuse
SafeFlowMatch	Flow Mat- ching	Explicite CBF	Obstacles ana- lytiques	Correction intra- génération	Non	Élevée en si- mulation
Notre ap- proche	Flow Mat- ching	Explicite : projection SDF-CBF	Nuage filtré persistant + SDF robot	Correction vitesse ar- ticulaire à l’exécution	Oui, via persis- tance	Élevée, mesurée expéri- mentale- ment

En somme, l’examen critique de la littérature scientifique met en évidence un compromis persistant entre l’expressivité des politiques génératives et la rigueur des algorithmes de préservation de la sécurité. Si le *Flow Matching* lissant et déterministe permet de capturer efficacement la multimodalité des mouvements experts tout en réduisant la latence propre aux processus de diffusion stochastique, son exécution en boucle ouverte au sein d’environnements réels et changeants demeure sujette aux incertitudes perceptives. Les méthodes contemporaines tentant d’incorporer des contraintes au cours de la phase de génération tendent à alourdir le temps de calcul ou à provoquer une dérive distributionnelle nuisible pour le succès de la tâche. Afin d’apporter une solution pragmatique à ces verrous technologiques, le Chapitre 3 détaillera la méthodologie de l’architecture découplée proposée dans ce travail. Nous y exposerons comment un pipeline de perception persistant s’articule de manière synchrone avec un filtre réactif basé sur une approximation par polynômes de Bernstein pour améliorer, en temps réel et sous des contraintes vérifiables localement, l’exécution sécurisée définie au

cœur de notre hypothèse de recherche.

CHAPITRE 3 MOTIVATION DE RECHERCHE ET OBJECTIFS

3.1 Contexte et problématique

3.2 Objectifs de recherche

L'intégration d'un filtre de sécurité SDF-CBF fondé sur un SDF local du robot permet de corriger en temps réel les vitesses nominales générées par un modèle Flow Matching, afin d'exécuter des trajectoires dans des environnements non vus tout en réduisant les risques de collision et en préservant l'intention de la trajectoire nominale.

3.3 Contexte et problématique

3.4 Portée et exclusions de la recherche

CHAPITRE 4 DESIGN DU SYSTÈME

4.2 Perception Sémantique et Reconstruction 3D

4.2.1 Extraction Sémantique

Parler de l'utilisation de VLM et de SAM 3 qui prends en entrée prompt de VLM pour détecter le target. Tu vas avoir un output mask qui va permettre de voir où se trouve le target au premier frame.

4.2.2 Suivi temporel de la cible

4.2.3 Projection RGB-D vers l'espace 3D

4.2.4 Séparation cible, obstacles et outil visible

4.2.5 Nettoyage spatial du nuage d'obstacles

4.3 Cartes Persistantes de la scène

4.3.1 Motivation : robustesse aux pertes de détection et occlusion

On veut pouvoir éviter les occlusions et les pertes de détection de SAM 3, en gardant en mémoire les surfaces des objets

4.3.2 Génération des cartes persistantes

4.3.3 Mise à jour des cartes persistantes

Conservation des points hors frame

Voxelisation de la map pour alléger les calculs

Suppression des doublons dans chaque Voxelisation

fusion avec la carte existante

jusqu'à la limite $\max_voxels$ (suppression des points par vieillissement temporel par la suite pour voir les points les plus proches et les nouveaux points)

Suppression des voxels supprimées si $z_{measure} - z_{voxel} > seuil$ (car la caméra voit là où le voxel était supposé caché)

4.4 Acquisition des données et représentation des démonstrations

Cette sous-section a comme objectif de présenter le protocole utilisé afin de collecter les données nécessaires pour entraîner le modèle d’IA générative ainsi que l’étape de prétraitement de ces données.

4.4.1 Protocol de collecte des démonstrations

Les données d’entraînement utilisée pour entraîner le modèle de Flow Matching ont été acquises de deux manières. La première d’entre elle est sur le robot et les capteurs réels et la seconde est issue de simulation avec la librairie Maniskill 3. Cette librairie a été choisie pour sa facilité d’utilisation et son moteur de calcul à l’état de l’art basé sur SAPIEN 3 [?]. Les deux méthodes utilisent des environnements quasi identiques afin de récolter les données de la même manière. Pour garantir un protocole reproductible, l’échantillonnage des configurations est défini dans deux repères imbriqués plutôt que de façon qualitative : le repère robot $\mathcal{F}_b$ (base du manipulateur, fixe) et un repère assiette $\mathcal{F}_p$ centré sur l’assiette. L’assiette est placée successivement aux 9 positions d’une grille 3×3 exprimées directement dans $\mathcal{F}_b$ (pas de XXX m). Pour chacune, la cible occupe 5 positions disposées en quinconce dans $\mathcal{F}_p$, chacune assortie d’une orientation aléatoire. Cette grille produit $9 \times 5 = 45$ configurations, chacune réalisée une fois. La Figure ?? illustre cet échantillonnage. À chaque configuration, l’outil descend verticalement vers la cible (axe $-z$ de $\mathcal{F}_b$) en maintenant les dents verticales et alignées sur l’axe principal de la cible. La simulation (*ManiSkill 3*) réutilise ce même schéma mais l’élargit par une randomisation de hauteur couvrant un surensemble continu des configurations réelles.

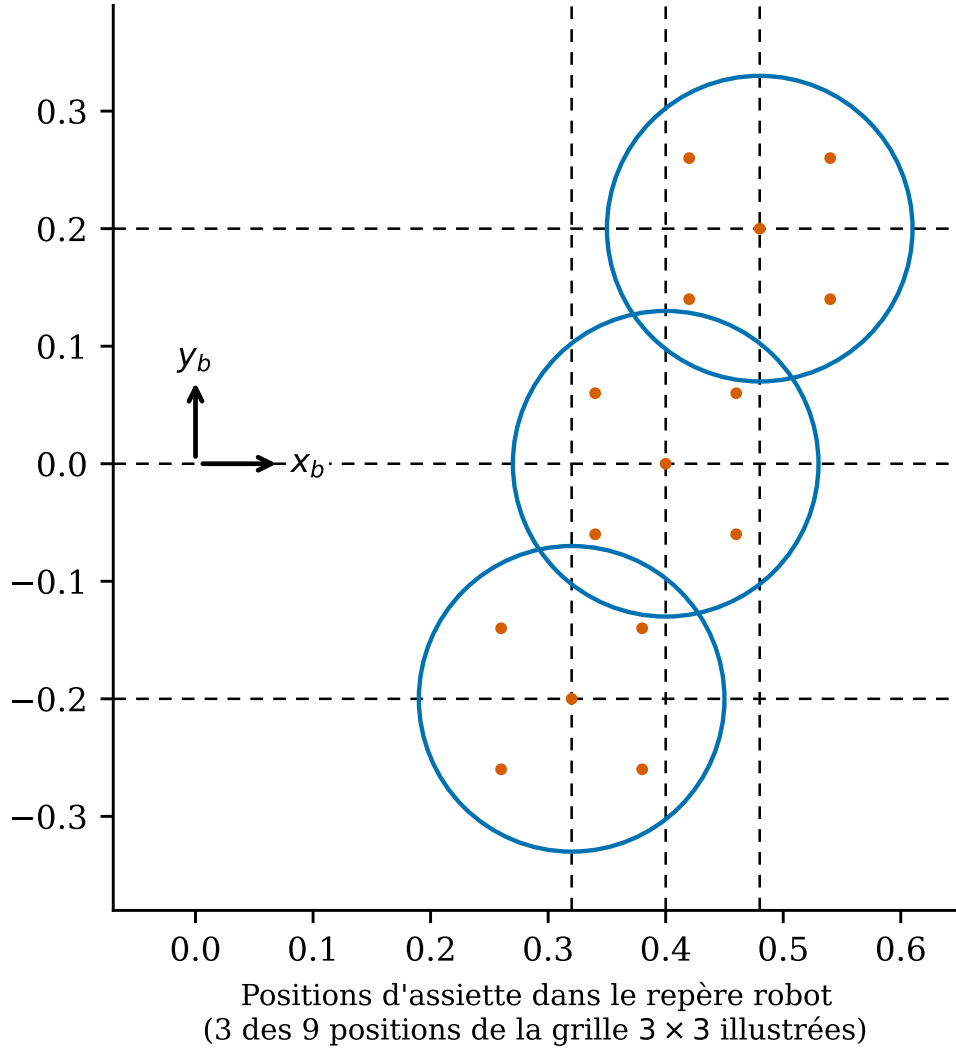


Figure 4.2 Protocole d'échantillonnage des cibles, vu de dessus dans le repère de base du robot $\mathcal{F}_b$. Les assiettes (26 cm de diamètre) se chevauchant, seules trois des neuf positions de la grille 3×3 sont illustrées (supérieur droit, centre, inférieur gauche), le centre de chacune étant marqué par une croix noire. Autour de chaque centre, les 5 points indiquent les positions de cible en quinconce.

Pour récolter les données réelles, le protocole détaillé suivant a été utilisé :

1. Installation de la caméra sur le support officiel de Franka au niveau de l'effecteur. Le support est en annexe
2. Positionnement des joints du robot à $\mathbf{q} = [0, -0.1259, 0, -2.1933, 0, 2.0648, 0.7855]$
3. Mettre le robot en mode Réflexe pour qu'il puisse être guider par un humain.
4. Réaliser la trajectoire robotique souhaitée pour aller chercher la cible et réaliser la

trajectoire à apprendre par le modèle.

5. Enregistrer les données dans un fichier JSON.

L'enregistrement des données se fait en utilisant 2 noeuds ROS distincts. Le premier noeud, écrit en C++ pour la rapidité d'exécution et de lecture, lis les données suivantes provenant du robot :

1. Pose du TCP dans le repère monde [1,7]. Les trois premières dimensions sont reliées à la position cartésienne et les 4 dernières à l'orientation sous la forme d'un quaternion.
2. Vitesse du TCP dans le repère monde [1,6]. Les trois premières position sont les vitesses cartésiennes et les 3 suivantes les vitesses angulaires.

L'ensemble de ces données sont publiées sous la forme de message ROS. Il sont par la suite suivies par un second noeud écrit en Python, qui suit aussi les données RGB-D de la caméra à l'effecteur. Ce noeud a la responsabilité d'enregistrer l'ensemble des données d'entraînement dans un fichier JSON. En plus des données du TCP, les données de la caméra ainsi que les données structurelles suivantes sont ajoutées afin d'organiser la base de données :

1. `step_number` : le numéro du jeu de données pour la trajectoire
2. `time_step` : le moment de la prise de données depuis le début de la trajectoire
3. `rgb` : le nom du fichier de la photo. Ce dernier est, par nomenclature, nommé `ee_rgb_step_{step_number}.png`.
4. `depth_npy` : le nom du fichier de la carte de profondeur. Ce dernier est, par nomenclature, nommé `ee_depth_step_{step_number}.npy`

La figure ?? représente l'architecture du système d'acquisition de données pour entraîner le modèle d'IA générative.

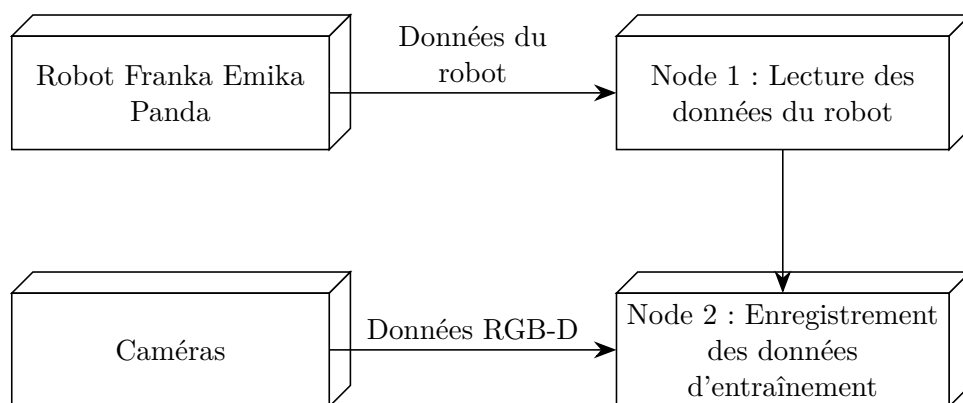


Figure 4.3 Architecture du système d'acquisition de données

Ces données sont ensuite enregistrées dans un dossier racine sous la forme d'un dossier nommé Trajectoire_X où X représente le numéro de la trajectoire enregistrée. Dans ce dossier chacun de ces dossiers se trouve un fichier JSON avec l'ensemble des données numériques et les noms des fichiers images ainsi qu'un dossier contenant l'ensemble des images et des fichiers de profondeurs. la représentation de l'architecture de l'enregistrement des fichiers est représentée sur la figure ?? :

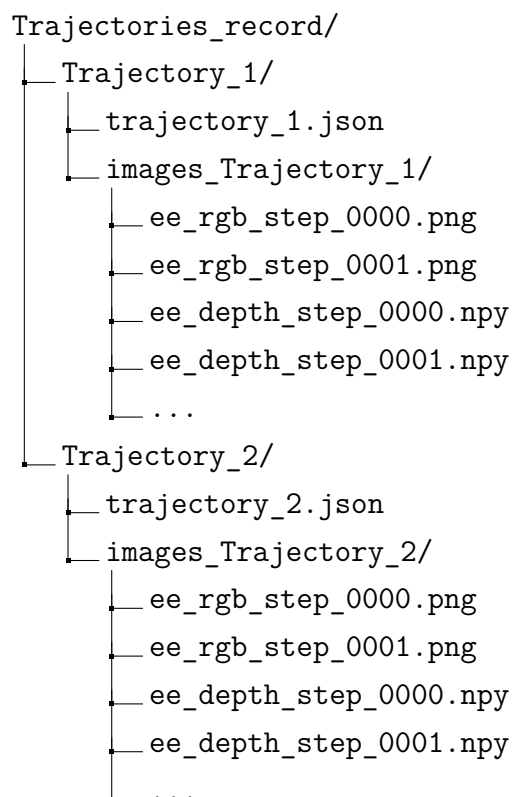


Figure 4.4 Directory structure of the PickPlace dataset execution tests.

4.4.2 Prétraitement des données

À partir des données enregistré en ??, plusieurs étapes de prétraitement sont réalisées hors ligne afin d'accélérer l'entraînement du modèle. Ces étapes sont listées et détaillées dans cette sous-section.

Détection avec un modèle de segmentation

La première étape consiste à détecter la cible dans la première image de chacune des trajectoires. Cette détection est réalisée avec le modèle à l'état de l'art Segment Anything 3

(SAM3) qui peut réaliser de la segmentation par prompt. Ainsi, une liste de cibles possibles est donnée en entrée au modèle et le masque de la cible dans l'image est obtenue en sortie. La liste en question est [*Pink block*, *Brocoli*, *Carot Slice*, *Chicken breast*, *Sushi*] Comme SAM3 prends que des images .jpg en entrée, il faut alors transformer la première image dans ce format. La raison de pourquoi on ne détecte pas la cible à chaque image est en raison de la limite physique du capteur de profondeur de la caméra utilisée qui est de 45 cm selon le détaillant. La figure ?? présente un exemple de détection d'un *Pink Block*.



Figure 4.5 Exemple de masque obtenu par le modèle de segmentation SAM3 pour le prompt *Pink Block*

Génération du nuage de point conditionnel

Considérant que le modèle de Flow Matching décrit dans la section ?? est conditionné sur les nuages de points de l'outil et de la cible, il est impératif de généré les nuages de points de chacun des étapes hors ligne afin d'accélérer le temps d'entraînement et de valider les données utilisées. Pour se faire, on utilise le masque généré à l'étape précédente, la carte de profondeur initiale, les angles dans les joints à chaque étape et le modèle CAO de l'outil.

La première étape consiste à générer un nuage de points de la cible dans le repère de la caméra. Pour y parvenir, une superposition du masque de la cible et de la carte de profondeur initiale est réalisée afin de conserver seulement les points de la cible. Par la suite, on calcul la position cartésiennes de chacun des points présents dans le masque en utilisant les équations

ci-dessous :

$$X_c = \frac{Z}{f_x}(u - c_x) \quad (4.1)$$

$$Y_c = \frac{Z}{f_y}(v - c_y) \quad (4.2)$$

$$Z_c = Z \quad (4.3)$$

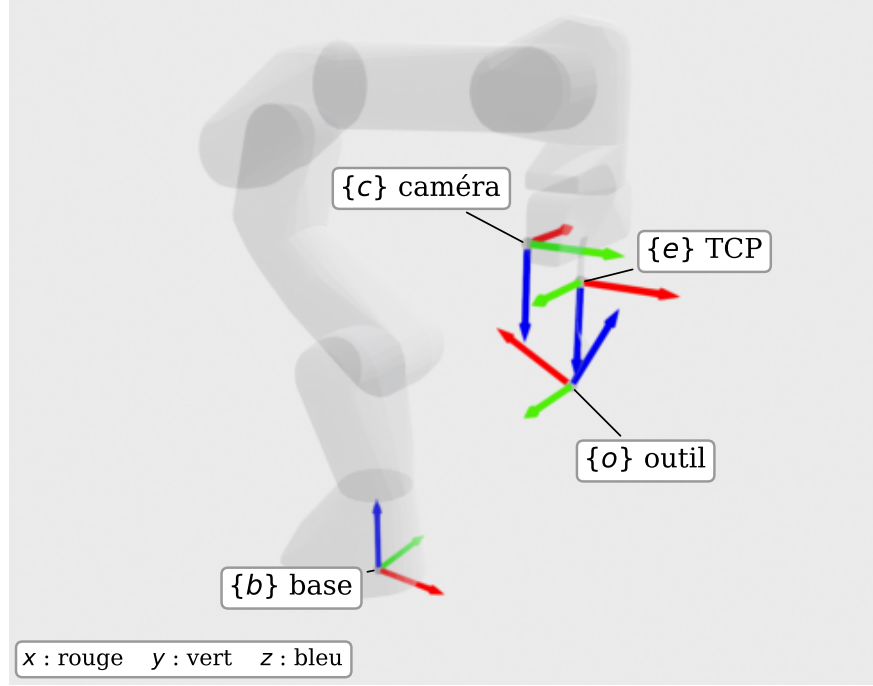


Figure 4.6 Représentation schématique des frames des différents repères utilisés

où f_x et f_y sont les longueurs focales de la caméra et c_x et c_y sont les coordonnées du point principal de la caméra. Les paramètres intrinsèques de la caméra D415 utilisée ont été obtenues au prêt du fabricant. Le nuage de points résultant est ensuite positionné dans le repère de la base du robot en utilisant la relation (??) où $\mathbf{T}_{cible}^b$ est la matrice de transformation entre la cible et la base du robot. Cette relation peut être déduite de la figure ?? où est représentée l'ensemble des repères nécessaires au système : $\{b\}$ le repère de la base du robot, $\{c\}$ pour le repère de la caméra au niveau de l'effecteur, $\{e\}$ pour le repère du TCP et $\{o\}$ pour le repère de l'outil utilisé. Ce dernier est finalement sous-échantillonner à 512 points en utilisant la stratégie d'échantillonnage par le point le plus éloigné.

$$\mathbf{T}_{cible}^b = \mathbf{T}_e^b \mathbf{T}_c^e \mathbf{T}_{cible}^c \quad (4.4)$$

La seconde étape consiste à générer le nuage de points de l'outil. En supposant que le modèle CAD de l'outil est disponible, il est possible d'obtenir son nuage de points en utilisant la librairie `trimesh` avec la fonction `trimesh.sample.sample_surface`. Pour positionner ce nuage de points, et connaissant la pose du repère de l'outil dans le repère du TCP du robot, l'équation (??) est utilisée.

$$\mathbf{T}_f^b = \mathbf{T}_e^b \mathbf{T}_f^e \quad (4.5)$$

à noter que la transformation $\mathbf{T}_f^e = \mathbf{I}$ dans le cas où l'outil est le TCP. Comme pour la cible, le nuage de point ainsi positionné est par la suite sous-échantionner à 512 points en utilisant la même stratégie. Les deux nuages de points résultants sont finalement fusionné ensemble et translaté au niveau du repère de la l'outil. Ce nuage de points final est ensuite enregistré sous le nom de `Merged_{step_number}` dans le dossier `Merged_Trajectory_X`. Finalement, les fichiers JSON provenant de l'étape précédente sont copiés dans leur dossier du jeu de données prétraités respectif et la pose de l'outil dans le repère monde $\{b\}$ y est ajoutée. L'architecture depuis le dossier racine du jeu de données ainsi créée est représentée à la figure ??.

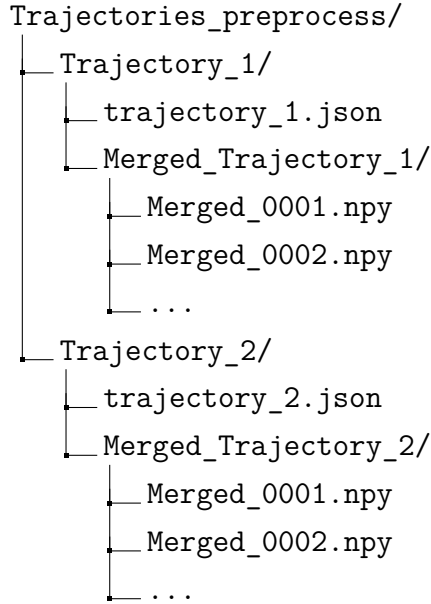


Figure 4.7 Structure du dossier du jeu de données des trajectoires prétraités

4.5 Génération de trajectoire par Flow Matching

Cette section traite de l'ensemble de l'architecture et des hyperparamètres utilisés pour entraîner le modèle de Flow Matching conditionnel afin de générer les trajectoires robotiques.

4.5.1 Formulation mathématique du Flow Matching conditionnel

Présenter la provenance et les mathématiques

$$\frac{d}{dt}\psi_t(x) = u_t(\psi_t(x)) \quad (4.6)$$

$$X_t = (1 - t)X_0 + tX_1 \quad (4.7)$$

$$\mathcal{L}(\theta) = \mathbb{E}_{t, X_0, X_1} \|u_t^\theta(X_t) - (X_1 - X_0)\|^2, \text{ où } t \sim U[0, 1], X_0 \sim \mathcal{N}(0, 1), X_1 \sim q \quad (4.8)$$

4.5.2 Architecture du modèle

Présenter les entrants, les extrants, l'architecture du modèle (EMA, normalisation, MLP pour encodeur, application du conditionnement des trajectoires), comment il résout l'ODE (avec approximation d'Euler) et comment j'obtiens les positions finales

$$\text{Mish}(x) = x \cdot \tanh(\ln(1 + e^x)) \quad (4.9)$$

4.5.3 Entraînement du modèle

Présentation des hyperparamètres choisis et de la Loss curve (finalement loss curve pas si importante comme on présente une méthodologie).

4.6 Conversion et exécution de la trajectoire nominale

4.6.1 Conversion outil - TCP

Mettre une image des repères TCP - Outil et de la matrice de Transformation et comment on fait pour calculer la pose du TCP à contrôler en fonction de la pose de l'outil

4.6.2 Cinématique inverse pour obtenir la trajectoire articulaire nominale

Comment est-ce qu'on fait la cinématique inverse d'un robot à 7 DOF ?

4.6.3 Retiming temporel de la trajectoire articulaire

Dire pourquoi on doit faire du retiming temporel : Parce que notre trajectoire n'a pas toujours la même dimension (distance entre les waypoints est différentes de chunk de trajectoires en

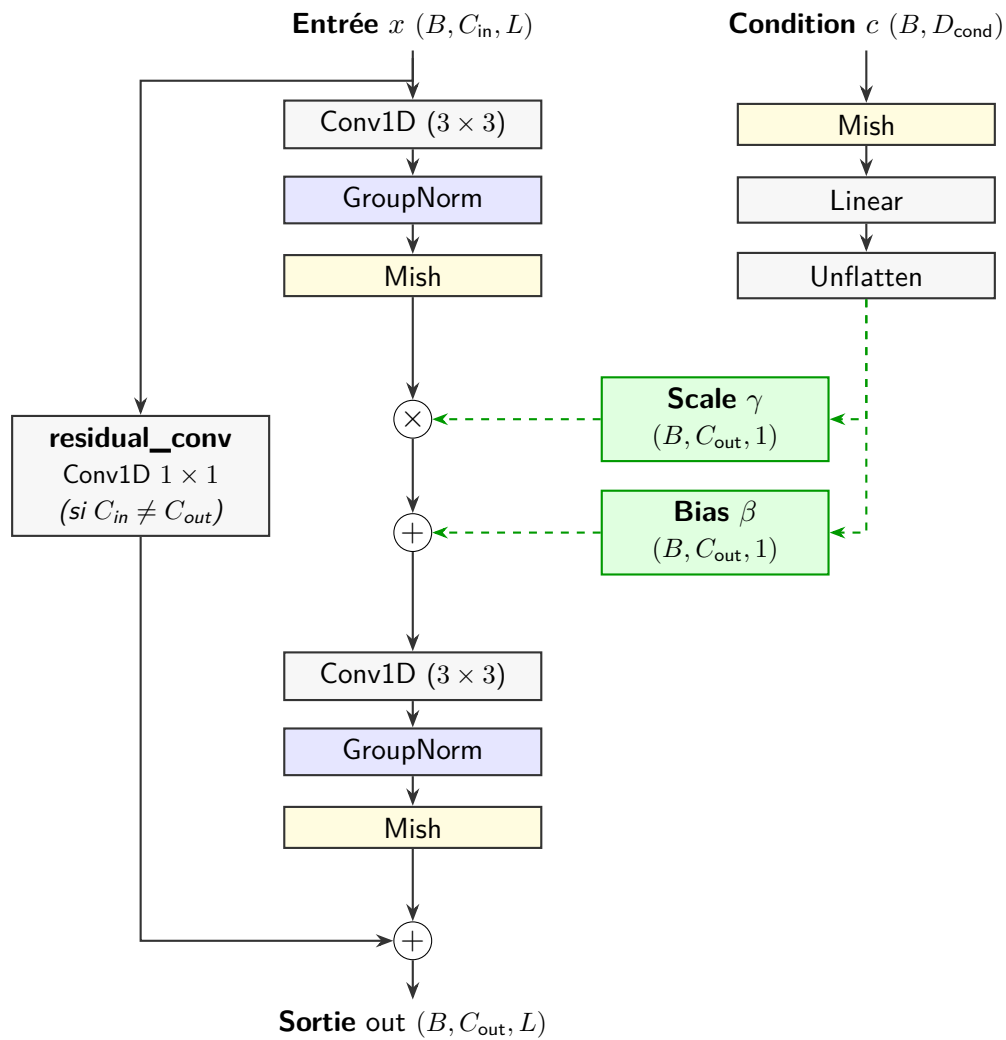


Figure 4.8 Architecture du module **ConditionalResidualBlock1D** avec un tracé orthogonal à angle droit unique vers les entrées latérales.

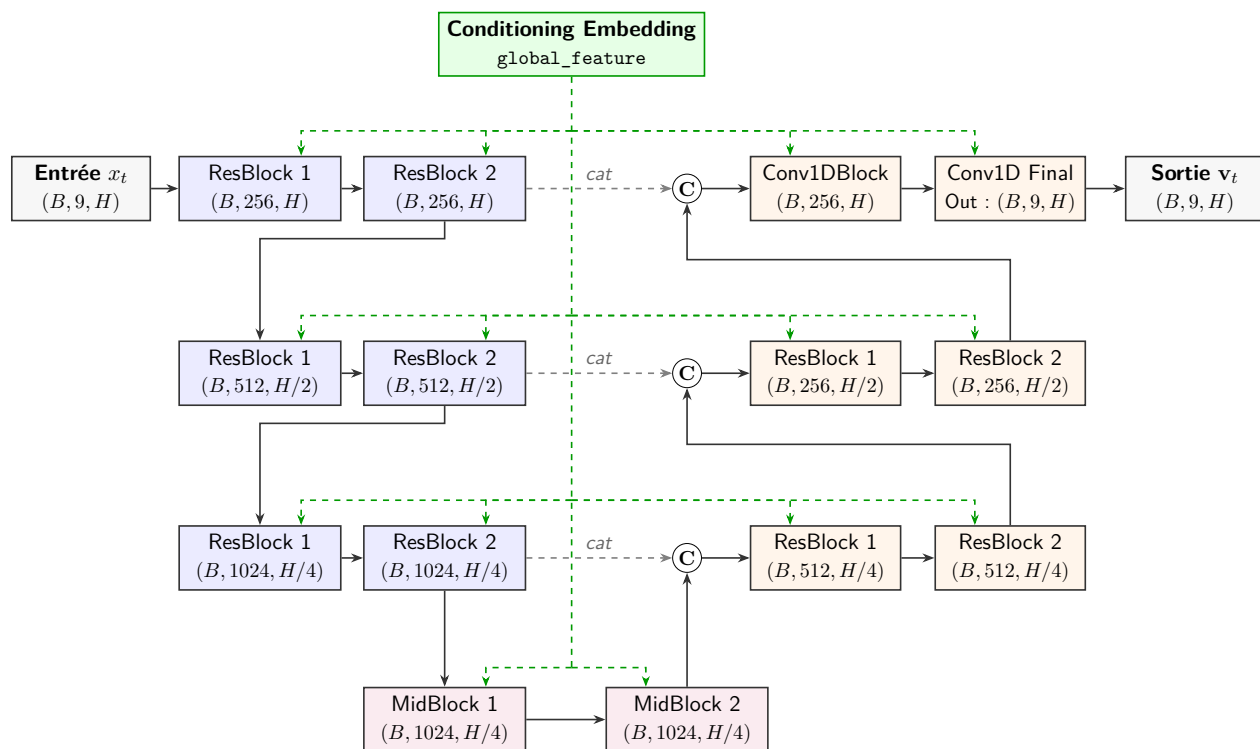


Figure 4.9 Architecture globale finale du ConditionalUnet1D au style épuré, corrigée de tout dépassement de tracé.

chunk de trajectoires), ce qui fait en sorte qu'on ne peut pas simplement dire de passer 10 points par seconde. Donc on doit calculer la vitesse pour avoir une vitesse constante de chunk en chunk. Spécifié que pour obtenir une vitesse constante, on calcul la vitesse en faisant la différentiation numérique de la position du système (mettre équation de la différentiation numérique pour obtenir la vitesse en position et en angle), puis on calcul la vitesse des joints avec la cinématique inverse.

En fait ce qui est fait, ce qu'on a la position des joints en faisant la cinématique inverse, puis on calcul la vitesse de chaque joint par cinématique inverse avec dérivation de degré 1. Après la cinématique inverse et le retiming, l'exécuteur de trajectoire interpole la trajectoire articulaire à la fréquence de commande et publie la vitesse feedforward associée :

$$\dot{\mathbf{q}}_{\text{ff}}(t) = \frac{\mathbf{q}_{i+1} - \mathbf{q}_i}{t_{i+1} - t_i}, \quad t \in [t_i, t_{i+1}]. \quad (4.10)$$

Cette vitesse représente l'intention instantanée de la trajectoire générée par *Flow Matching*. Grâce au retiming, sa norme est approximativement constante malgré des points de passage non équidistants, ce qui fournit au filtre de sécurité une référence régulière plutôt qu'une différentiation numérique bruitée de la consigne de position.

4.6.4 Interpolation temporelle et exécution nominale

Calcul de cinématique inverse pour obtenir $\mathbf{q}$ Présentation de la méthode pour calculer la cinématique inverse (code C++ avec Pinocchio), puis retiming pour conserver une vitesse constante malgré des waypoints non-equidistants

4.7 Filtre de sécurité réactif SDF-CBF

Le modèle génératif est entraîné sur des démonstrations sans obstacle et exécuté en boucle ouverte : il n'offre aucune garantie vis-à-vis d'un environnement non vu à l'entraînement. Plutôt que de contraindre le processus de génération (au prix d'une dérive distributionnelle et d'un coût de calcul accru, voir le Chapitre ??), la sécurité est déléguée à un filtre réactif agissant sur la vitesse articulaire à la fréquence de commande. Formellement, à chaque cycle de commande, le filtre résout le problème de projection

$$\dot{\mathbf{q}}^* = \arg \min_{\dot{\mathbf{q}} \in \mathbb{R}^7} \frac{1}{2} \left\| \dot{\mathbf{q}} - \dot{\mathbf{q}}_{\text{base}} \right\|^2 \quad \text{s.c.} \quad \nabla h(\mathbf{q})^\top \dot{\mathbf{q}} \geq b(h(\mathbf{q})), \quad (4.11)$$

où $\mathbf{q} \in \mathbb{R}^7$ est la configuration articulaire mesurée : la commande retenue est la vitesse la plus proche de la vitesse nominale $\dot{\mathbf{q}}_{\text{base}}$, l'intention du modèle génératif, parmi celles que la condition de sécurité autorise. Ce problème comporte trois ingrédients, qui structurent la section. La fonction barrière $h(\mathbf{q})$, qui mesure la distance signée entre le robot et les points obstacles $\mathbf{p}_i \in \mathbb{R}^3$ de la carte persistante, repose sur un modèle de distance du robot appris hors ligne (Section ??), à partir duquel elle est assemblée en ligne (Section ??). Le second membre $b(h)$ et la vitesse nominale $\dot{\mathbf{q}}_{\text{base}}$, sont définis à la Section ?. La Section ?? résout le problème en forme analytique et décrit le post-traitement qui transforme cette solution en commande exécutable $\dot{\mathbf{q}}_{\text{final}}$. Enfin, la Section ?? généralise (??) à plusieurs contraintes simultanées et délimite la portée des garanties obtenues.

4.7.1 Modèle de distance du robot appris hors ligne

Le filtre doit connaître, en quelques millisecondes et pour des milliers de points simultanément, la distance signée entre tout point de l'espace et la surface du robot — ainsi que son gradient. Suivant l'approche *Robot Distance Fields* [?], également exploitée pour l'évitement réactif de collisions dans un cadre de commande à priorité de tâches [?], le SDF de chaque lien l est approximé par une base tensorielle de polynômes de Bernstein dans un repère normalisé propre au lien. La géométrie maillée du lien est d'abord centrée et mise à l'échelle :

$$\bar{\mathbf{x}} = \frac{\mathbf{x} - \mathbf{o}_l}{s_l}, \quad \mathbf{o}_l = \text{centre de la boîte englobante}, \quad s_l = \max_{\mathbf{v} \in \mathcal{V}_l} \|\mathbf{v} - \mathbf{o}_l\|, \quad (4.12)$$

où $\mathcal{V}_l$ est l'ensemble des sommets du maillage : le lien est ainsi contenu dans le cube unitaire $[-1, 1]^3$. Sur ce domaine, le SDF normalisé est une combinaison linéaire de n^3 fonctions de base :

$$\bar{d}_l(\bar{\mathbf{x}}) = \sum_{a,b,c=0}^{n-1} w_{abc}^{(l)} B_a(u_x) B_b(u_y) B_c(u_z) = \Phi(\bar{\mathbf{x}}) \mathbf{w}_l \quad (4.13)$$

$$B_a(u) = \binom{n-1}{a} u^a (1-u)^{n-1-a} \quad (4.14)$$

$$\mathbf{u} = \frac{\bar{\mathbf{x}} + 1}{2} \quad (4.15)$$

et la distance métrique s'en déduit par $d_l = s_l \bar{d}_l$. Les liens protégés (liens 4 à 7, main et ouil) utilisent $n = 12$ fonctions par axe, soit 1728 coefficients par lien ; les liens proximaux, jamais menacés dans les scénarios considérés, restent à $n = 8$. Trois propriétés motivent ce choix de représentation : le modèle est polynomial, donc $\mathcal{C}^\infty$ et de gradient analytique exact ; il est linéaire en ses paramètres $\mathbf{w}_l$, ce qui rend l'ajustement convexe ; et son évaluation se réduit

à des produits tensoriels, massivement parallélisables sur GPU.

L'apprentissage des poids procède en deux étapes par lien. Un jeu d'étiquettes est d'abord généré sur le maillage, suivant le protocole d'échantillonnage de [?] : 8×10^5 points concentrés près de la surface (balayages virtuels, signe déterminé par les normales) et autant de points uniformes dans le cube, chacun étiqueté de sa distance signée exacte ; les étiquettes négatives situées hors de la boîte englobante du lien, artefacts du calcul de signe, sont écartées. La linéarité du modèle permet alors un ajustement par *moindres carrés récurrents* : à chaque itération, un lot de points Φ_k d'étiquettes $\mathbf{d}_k$ met à jour les poids en forme close,

$$\mathbf{K}_k = \mathbf{B}_{k-1} \Phi_k^\top (\mathbf{I} + \Phi_k \mathbf{B}_{k-1} \Phi_k^\top)^{-1} \quad (4.16)$$

$$\mathbf{B}_k = \mathbf{B}_{k-1} - \mathbf{K}_k \Phi_k \mathbf{B}_{k-1} \quad (4.17)$$

$$\mathbf{w}_k = \mathbf{w}_{k-1} + \mathbf{K}_k (\mathbf{d}_k - \Phi_k \mathbf{w}_{k-1}) \quad (4.18)$$

avec $\mathbf{B}_0 = 10^2 \mathbf{I}$ (régularisation de type *ridge*). Cette régression de valeur, qui correspond à l'apprentissage original de [?], fournit une distance précise mais ne contraint pas le gradient : près de la surface, sa direction reste bruitée et sa norme s'écarte de l'unité.

Or le filtre de sécurité consomme précisément ce gradient : ∇h fixe la *direction* de la correction de la projection (??) et sa *norme* en fixe l'amplitude. Une seconde étape de raffinement géométrique est donc appliquée. Un jeu de supervision dédié de 20 000 *rayons normaux vérifiés* est construit : des points de surface sont déplacés le long de la normale de leur facette d'un décalage δ tiré aléatoirement, concentré à 75 % dans la coquille critique $[1, 10]$ mm (cohérente avec $d_{\text{safe}} = 15$ mm) et étendu jusqu'à 50 mm. Chaque candidat n'est retenu que s'il est vérifié contre le maillage exact : le point doit être extérieur, sa distance exacte recalculée doit coïncider avec le décalage à 0,25 mm près, et la direction exacte du gradient ne doit pas s'écarter de plus de 5° de la normale de départ — ce qui élimine les rayons corrompus par l'axe médian ou la courbure. Chaque rayon fournit ainsi un triplet exact (point, distance, direction sortante $\hat{\mathbf{n}}$). Les poids sont alors affinés par descente de gradient (Adam, recuit en cosinus) sur la fonction de coût

$$\begin{aligned} \mathcal{L} = & \mathcal{L}_{\text{près}} + \mathcal{L}_{\text{unif}} + \lambda_{\text{ray}} \mathcal{L}_{\text{ray}} + \lambda_{\text{eik}} \mathbb{E}[(\|\nabla \bar{d}\| - 1)^2] \\ & + \mathbb{E}\left[\omega\left(\lambda_{\text{dir}} (1 - \widehat{\nabla d} \cdot \hat{\mathbf{n}}) + \lambda_{\text{vec}} \|\nabla \bar{d} - \hat{\mathbf{n}}\|^2\right)\right] \end{aligned} \quad (4.19)$$

où les trois premiers termes sont les erreurs quadratiques de valeur (points près de la surface, uniformes et rayons, $\lambda_{\text{ray}} = 2$), le terme eikonal ($\lambda_{\text{eik}} = 0,05$) impose $\|\nabla \bar{d}\| = 1$ partout, et les

deux derniers alignent la direction du gradient sur la direction exacte des rayons ($\lambda_{\text{dir}} = 0,2$, $\lambda_{\text{vec}} = 0,25$), avec une pondération $\omega = 1 + 4e^{-\bar{d}/\bar{\delta}}$ qui concentre l'effort près de la surface. Le gradient $\nabla \bar{d}$ étant la dérivée analytique d'un polynôme, aucune approximation numérique n'intervient.

Enfin, le passage des modèles validés d'ordre $n = 8$ à l'ordre $n = 12$ s'effectue par *élévation de degré* exacte de Bernstein, appliquée séparément sur chaque axe du produit tensoriel :

$$\mathbf{w}' = (\mathbf{E} \otimes \mathbf{E} \otimes \mathbf{E}) \mathbf{w}, \quad E_{ij} = \frac{\binom{m}{j} \binom{m'-m}{i-j}}{\binom{m'}{i}},$$

où m et m' sont les degrés initial et final : la fonction représentée est rigoureusement inchangée, ce qui fournit au raffinement une initialisation qui préserve le modèle déjà validé. Les modèles finaux résultent de 80 itérations de moindres carrés récursifs suivies de 1500 époques de raffinement géométrique. La précision de distance, la direction du gradient et la propriété eikonale qui en résultent sont caractérisées expérimentalement au Chapitre ??.

4.7.2 Fonction barrière

À l'exécution, chaque point obstacle $\mathbf{p}_i$ est ramené dans le repère normalisé de chaque lien protégé via la cinématique directe, puis évalué par le modèle appris :

$$\mathbf{x}_{l,i}(\mathbf{q}) = \mathbf{T}_l(\mathbf{q})^{-1} \mathbf{p}_i, \quad d_{l,i}(\mathbf{q}) = s_l \Phi\left(\frac{\mathbf{x}_{l,i} - \mathbf{o}_l}{s_l}\right) \mathbf{w}_l.$$

Pour les points situés hors du domaine d'approximation $[-1, 1]^3$, la coordonnée normalisée est bornée au cube et la distance euclidienne au point borné est ajoutée — une correction conservatrice qui prolonge le modèle hors de son domaine d'entraînement. Cette formulation fournit une distance différentiable par rapport à $\mathbf{q}$, interrogeable en parallèle sur GPU pour l'ensemble des points obstacles.

Évaluer cette distance sur la totalité de la carte persistante à chaque cycle serait inutilement coûteux : seuls les points les plus proches du robot peuvent activer la contrainte. La carte est donc d'abord élaguée par un préfiltre sphérique autour des liens protégés, puis les K points de plus faible SDF corps-complet sont conservés :

$$\mathcal{P}_{\text{CBF}} = \text{TopK}_K \left(\min_l d_l(\mathbf{q}, \mathbf{p}_i) \right), \quad K = 100.$$

Cette sélection est mise à jour à une fréquence inférieure à celle de la boucle de commande (30 Hz contre 100 Hz). L'obsolescence maximale de l'ensemble actif qui en résulte (≈ 33 ms)

est prise en compte dans le budget de la marge de sécurité d_{safe} (Section ??).

Les distances $d_{l,i}$ des points retenus sont alors agrégées en une fonction barrière scalaire. Pour chaque lien, le minimum sur les points obstacles est approché par un soft-min de température α , formulé de manière numériquement stable autour de $m_l = \min_i d_{l,i}$:

$$h_l(\mathbf{q}) = m_l - \alpha \log \left[\sum_{i=1}^N \exp \left(-\frac{d_{l,i} - m_l}{\alpha} \right) \right] - d_{\text{safe}}.$$

La barrière globale correspond au lien le plus critique, et l'ensemble sûr est défini par :

$$h(\mathbf{q}) = \min_l h_l(\mathbf{q}), \quad \mathcal{C} = \{\mathbf{q} \mid h(\mathbf{q}) \geq 0\}.$$

Le soft-min introduit un biais conservateur borné par $\alpha \log N$; en notant h_{soft} la valeur brute issue du soft-min, la valeur compensée $h_{\text{corr}} = h_{\text{soft}} + \alpha \log N$ est utilisée pour l'interprétation des mesures — c'est sous ces deux noms que la barrière apparaît au Chapitre ??. Ce terme étant indépendant de $\mathbf{q}$, le gradient n'est pas affecté.

4.7.3 Gradient analytique de la contrainte

La projection (??) repose entièrement sur le gradient $\nabla_{\mathbf{q}} h$, qui en fixe à la fois la direction et l'amplitude. On l'obtient en forme close, par dérivation exacte de chaque maillon de la chaîne, ce qui en garantit le coût borné qu'exige le temps réel. Pour un point obstacle donné, la distance résulte de la composition de trois applications,

$$\mathbf{q} \xrightarrow{\text{cinématique}} \mathbf{x}_{l,i} \xrightarrow{\text{normalisation}} \mathbf{u} \xrightarrow{\text{SDF de Bernstein}} \bar{d}_l,$$

toutes trois dérivables analytiquement. Les différentielles correspondantes sont établies successivement, puis recombinaées par la règle de chaîne et agrégées à travers le soft-min.

Tout d'abord, en ce qui concerne la différentielle cinématique, l'obstacle est fixe dans le repère de base $\mathcal{F}_b$ ($\dot{\mathbf{p}}_i = \mathbf{0}$), mais paraît mobile dans le repère $\mathcal{F}_l$ que les articulations déplacent. Sa position locale $\mathbf{x}_{l,i} = {}^b\mathbf{R}_l^\top (\mathbf{p}_i - {}^b\mathbf{d}_l)$ vérifie la relation de fermeture $\mathbf{p}_i = {}^b\mathbf{d}_l + {}^b\mathbf{R}_l \mathbf{x}_{l,i}$, où ${}^b\mathbf{R}_l(\mathbf{q})$ et ${}^b\mathbf{d}_l(\mathbf{q})$ sont la rotation et la translation de $\mathbf{T}_l(\mathbf{q})$. En dérivant cette relation par rapport au temps et en annulant $\dot{\mathbf{p}}_i$:

$$\mathbf{0} = {}^b\dot{\mathbf{d}}_l + {}^b\dot{\mathbf{R}}_l \mathbf{x}_{l,i} + {}^b\mathbf{R}_l \dot{\mathbf{x}}_{l,i}.$$

Les blocs linéaire et angulaire de la Jacobienne géométrique du lien, ${}^b\mathbf{J}_{L,l}$ et ${}^b\mathbf{J}_{A,l}$, relient ces dérivées à la vitesse articulaire par ${}^b\dot{\mathbf{d}}_l = {}^b\mathbf{J}_{L,l}\dot{\mathbf{q}}$ et ${}^b\dot{\mathbf{R}}_l = \left[{}^b\mathbf{J}_{A,l}\dot{\mathbf{q}} \right]_{\times} {}^b\mathbf{R}_l$. En les substituant et

en isolant le terme de vitesse locale :

$${}^b\mathbf{R}_l \dot{\mathbf{x}}_{l,i} = -{}^b\mathbf{J}_{L,l} \dot{\mathbf{q}} - \left[{}^b\mathbf{J}_{A,l} \dot{\mathbf{q}} \right]_{\times} {}^b\mathbf{R}_l \mathbf{x}_{l,i}.$$

Une multiplication à gauche par ${}^b\mathbf{R}_l^{\top}$, suivie des identités du produit vectoriel $\mathbf{R}^{\top}[\mathbf{a}]_{\times}\mathbf{R} = [\mathbf{R}^{\top}\mathbf{a}]_{\times}$ et $[\mathbf{a}]_{\times}\mathbf{b} = -[\mathbf{b}]_{\times}\mathbf{a}$, isole la Jacobienne recherchée $\mathbf{J}_{x_{l,i}} = \partial\mathbf{x}_{l,i}/\partial\mathbf{q}$, telle que $\dot{\mathbf{x}}_{l,i} = \mathbf{J}_{x_{l,i}} \dot{\mathbf{q}}$:

$$\boxed{\mathbf{J}_{x_{l,i}}(\mathbf{q}) = -{}^b\mathbf{R}_l^{\top} {}^b\mathbf{J}_{L,l} + \left[\mathbf{x}_{l,i} \right]_{\times} {}^b\mathbf{R}_l^{\top} {}^b\mathbf{J}_{A,l}}$$

le premier terme traduisant la translation du repère du lien, le second sa rotation. La matrice antisymétrique associée au produit vectoriel y vaut

$$\left[\mathbf{x}_{l,i} \right]_{\times} = \begin{bmatrix} 0 & -x_3 & x_2 \\ x_3 & 0 & -x_1 \\ -x_2 & x_1 & 0 \end{bmatrix}, \quad \mathbf{x}_{l,i} = (x_1, x_2, x_3)^{\top}.$$

La deuxième application, la normalisation qui ramène l'obstacle dans le cube d'apprentissage, $\mathbf{u} = (\mathbf{x}_{l,i} - \mathbf{o}_l)/(2s_l) + \frac{1}{2}\mathbf{1}$, est affine ; sa Jacobienne est la matrice diagonale constante

$$\frac{\partial\mathbf{u}}{\partial\mathbf{x}_{l,i}} = \frac{1}{2s_l} \mathbf{I}_3.$$

La troisième différentielle, le gradient du SDF dans l'espace normalisé, découle de la dérivation exacte de la base tensorielle. La dérivée d'un polynôme de Bernstein de degré $n-1$ se ramène à la base de degré immédiatement inférieur par la règle d'abaissement

$$\frac{dB_a(u)}{du} = (n-1) \left(B_{a-1}^{(n-2)}(u) - B_a^{(n-2)}(u) \right),$$

d'où, en dérivant le produit tensoriel selon chacun des trois axes, le gradient $\nabla_{\mathbf{u}} \bar{d}_l \in \mathbb{R}^3$:

$$\nabla_{\mathbf{u}} \bar{d}_l = \begin{bmatrix} \sum_{a,b,c} w_{abc}^{(l)} B'_a(u_x) B_b(u_y) B_c(u_z) \\ \sum_{a,b,c} w_{abc}^{(l)} B_a(u_x) B'_b(u_y) B_c(u_z) \\ \sum_{a,b,c} w_{abc}^{(l)} B_a(u_x) B_b(u_y) B'_c(u_z) \end{bmatrix}.$$

La composition des trois différentielles, en tenant compte de la mise à l'échelle métrique $d_{l,i} = s_l \bar{d}_l$, fournit le gradient articulaire d'un point obstacle : le facteur s_l se simplifie avec celui de la normalisation, de sorte que le gradient spatial métrique se réduit à $\nabla_{\mathbf{x}} d_{l,i} = \frac{1}{2} \nabla_{\mathbf{u}} \bar{d}_l$. Plutôt que d'assembler explicitement la Jacobienne $\mathbf{J}_{x_{l,i}}$ pour chacun des points —

ce qui matérialiserait un tenseur coûteux —, on applique les mêmes identités du produit vectoriel pour réécrire $\nabla_{\mathbf{q}} d_{l,i} = \mathbf{J}_{x_{l,i}}^\top \nabla_{\mathbf{x}} d_{l,i}$ directement dans le repère de base. En notant $\mathbf{g}_{l,i} = {}^b\mathbf{R}_l \nabla_{\mathbf{x}} d_{l,i}$ le gradient spatial exprimé dans $\mathcal{F}_b$ et $\mathbf{r}_{l,i} = \mathbf{p}_i - {}^b\mathbf{d}_l$ le vecteur reliant l'origine du repère du lien à l'obstacle :

$$\nabla_{\mathbf{q}} d_{l,i} = - {}^b\mathbf{J}_{L,l}^\top \mathbf{g}_{l,i} - {}^b\mathbf{J}_{A,l}^\top (\mathbf{r}_{l,i} \times \mathbf{g}_{l,i}).$$

Cette forme, retenue dans l'implémentation, ne requiert que deux produits matrice-vecteur et un produit vectoriel par lien, et évite de construire la Jacobienne point par point.

Il reste à propager ce résultat à travers l'agrégation soft-min de la barrière. La dérivation du Log-Sum-Exp fait apparaître les poids du soft-max, ω_i , qui agissent comme un sélecteur lisse de la contrainte active :

$$\omega_i = \frac{\exp(-d_{l,i}/\alpha)}{\sum_{j=1}^N \exp(-d_{l,j}/\alpha)} = \text{softmax}_i(-d_{l,i}/\alpha),$$

et le gradient de la barrière du lien actif l^* est la combinaison convexe correspondante :

$$\nabla_{\mathbf{q}} h = \sum_{i=1}^N \omega_i \nabla_{\mathbf{q}} d_{l^*,i} = \sum_{i=1}^N \omega_i \mathbf{J}_{x_{l^*,i}}(\mathbf{q})^\top \nabla_{\mathbf{x}} d_{l^*,i}.$$

Chaque facteur étant la dérivée exacte d'un polynôme ou d'une transformation rigide, ce gradient est obtenu en forme close, sans aucune approximation numérique. Le soft-min sur les points lisse la direction du gradient en pondérant les points les plus proches, au prix du biais déjà mentionné ; le minimum strict sur les liens la dirige en revanche vers le lien réellement le plus menacé.

Tableau 4.1 Comparaison des performances et des erreurs de gradient.

Links	Analytique	différentiation numérique	Grad max error
1	5.596 ± 0.413	7.180 ± 0.113	5.960×10^{-8}
2	5.549 ± 0.345	8.010 ± 0.389	2.235×10^{-7}
3	5.432 ± 0.149	8.836 ± 0.457	2.373×10^{-7}
4	5.444 ± 0.106	9.789 ± 0.127	2.373×10^{-7}
5	5.538 ± 0.398	10.349 ± 0.321	2.980×10^{-7}
6	7.644 ± 0.156	12.257 ± 0.517	2.980×10^{-7}
7	7.792 ± 0.315	12.899 ± 0.288	2.980×10^{-7}
8	7.698 ± 0.172	13.479 ± 0.233	2.980×10^{-7}

4.7.4 Contrainte de sécurité et vitesse nominale

Le problème (??) requiert encore ses deux entrées : le second membre $b(h)$, qui délimite les vitesses admissibles, puis la vitesse nominale $\dot{\mathbf{q}}_{\text{base}}$, centre de l'objectif. Pour la première, la condition CBF classique impose $\nabla h^\top \dot{\mathbf{q}} \geq -\kappa h$. Avec une barrière issue de la perception, le terme linéaire κh présente deux inconvénients pratiques : loin de l'obstacle il autorise des vitesses d'approche arbitrairement grandes, et en violation il exige des vitesses de récupération proportionnelles à la pénétration, amplifiant le bruit de h . La contrainte implémentée sature ces deux régimes :

$$\nabla h(\mathbf{q})^\top \dot{\mathbf{q}} \geq b(h), \quad b(h) = m_{\text{CBF}} + \begin{cases} -v_{\text{in}} \min(h/h_{\text{act}}, 1), & h > 0, \\ v_{\text{rec}} \min(-h/d_{\text{rec}}, 1), & h \leq 0, \end{cases}$$

la contrainte n'étant évaluée que dans la bande d'activation $h \leq h_{\text{act}}$. Loin de la frontière, la vitesse d'approche normale est ainsi bornée par v_{in} ; à la frontière ($h = 0$), toute vitesse normale entrante est interdite ; en violation, une vitesse de sortie bornée par v_{rec} est exigée. Le terme de récupération proactif est fixé à $v_{\text{rec}} = 0$: la sortie de violation est déléguée à l'étape de réparation décrite à la Section ??, dont le gain κ_{rec} joue ce rôle. Cette formulation appartient à la famille $\nabla h^\top \dot{\mathbf{q}} \geq -\gamma(h)$ avec γ une fonction de classe $\mathcal{K}$ saturée.

La seconde entrée du problème (??) est la vitesse que le système *souhaite* exécuter. Celle-ci combine le feedforward retimé $\dot{\mathbf{q}}_{\text{ff}}$ (Section ??) et une correction de suivi proportionnelle saturée :

$$\dot{\mathbf{q}}_{\text{fb}}^0 = \text{sat}\left(K_p(\mathbf{q}_{\text{nom}} - \mathbf{q}_{\text{cur}}), \dot{q}_{\text{fb,max}}\right).$$

À proximité de la frontière, ce terme de suivi est cependant restreint à sa composante tangentielle d'avancement. En effet, lorsque le filtre dévie le robot de la trajectoire nominale pour des raisons de sécurité, un retour proportionnel complet pointe directement vers la trajectoire — c'est-à-dire contre la correction de sécurité — et les deux termes s'annulent en régime permanent, immobilisant le robot contre la frontière. Avec $\hat{\mathbf{t}} = \dot{\mathbf{q}}_{\text{ff}}/\|\dot{\mathbf{q}}_{\text{ff}}\|$ la direction d'avancement nominale :

$$\dot{\mathbf{q}}_{\text{fb}} = \begin{cases} \max(0, \hat{\mathbf{t}}^\top \dot{\mathbf{q}}_{\text{fb}}^0) \hat{\mathbf{t}}, & h \leq h_{\text{act}}, \\ \dot{\mathbf{q}}_{\text{fb}}^0, & \text{sinon.} \end{cases}$$

La vitesse nominale brute $\dot{\mathbf{q}}_{\text{raw}} = \dot{\mathbf{q}}_{\text{ff}} + \dot{\mathbf{q}}_{\text{fb}}$ est ensuite lissée par un filtre passe-bas de constante τ_{nom} :

$$\gamma_{\text{nom}} = \frac{\Delta t}{\Delta t + \tau_{\text{nom}}}, \quad \dot{\mathbf{q}}_{\text{base},k} = (1 - \gamma_{\text{nom}}) \dot{\mathbf{q}}_{\text{base},k-1} + \gamma_{\text{nom}} \dot{\mathbf{q}}_{\text{raw},k},$$

puis saturée aux limites de vitesse articulaire et annulée sous un seuil de maintien (évitant toute dérive en station). Le résultat $\dot{\mathbf{q}}_{\text{base}}$ est la *vitesse nominale filtrée* : c'est l'entrée du filtre de sécurité, et la référence par rapport à laquelle toutes les interventions de celui-ci sont mesurées au Chapitre ??.

4.7.5 De la vitesse nominale à la commande sécurisée

La commande sécurisée est construite à partir d'une vitesse nominale articulaire, puis modifiée uniquement lorsque la contrainte de sécurité devient active. L'exécuteur de trajectoire fournit d'abord une vitesse feedforward retimée $\dot{\mathbf{q}}_{\text{ff}}$, qui représente l'intention instantanée de la trajectoire générée. À cette vitesse est ajoutée une correction de suivi proportionnelle bornée $\dot{\mathbf{q}}_{\text{fb}}$, calculée à partir de l'erreur entre la consigne nominale courante et la configuration mesurée. La vitesse nominale brute est donc

$$\dot{\mathbf{q}}_{\text{raw}} = \dot{\mathbf{q}}_{\text{ff}} + \dot{\mathbf{q}}_{\text{fb}}.$$

Afin d'éviter les discontinuités dues à l'interpolation temporelle des points de passage, cette vitesse est filtrée par un filtre passe-bas exponentiel de constante de temps τ_{nom} :

$$\mathcal{F}_{\tau_{\text{nom}}}[\dot{\mathbf{q}}_{\text{raw}}]_k = (1 - \gamma_{\text{nom}})\mathcal{F}_{\tau_{\text{nom}}}[\dot{\mathbf{q}}_{\text{raw}}]_{k-1} + \gamma_{\text{nom}}\dot{\mathbf{q}}_{\text{raw},k},$$

avec

$$\gamma_{\text{nom}} = \frac{\Delta t}{\Delta t + \tau_{\text{nom}}}.$$

La vitesse nominale transmise au filtre CBF est ensuite saturée aux limites articulaires :

$$\dot{\mathbf{q}}_{\text{base}} = \text{sat}(\mathcal{F}_{\tau_{\text{nom}}}[\dot{\mathbf{q}}_{\text{raw}}]).$$

La projection CBF principale calcule ensuite la vitesse admissible la plus proche de $\dot{\mathbf{q}}_{\text{base}}$:

$$\dot{\mathbf{q}}_{\text{proj}} = \arg \min_{\dot{\mathbf{q}}} \frac{1}{2} \|\dot{\mathbf{q}} - \dot{\mathbf{q}}_{\text{base}}\|^2 \quad \text{s.c.} \quad \nabla h^\top \dot{\mathbf{q}} \geq b(h).$$

La sortie de cette projection est $\dot{\mathbf{q}}_{\text{proj}}$. La correction de sécurité $\Delta \dot{\mathbf{q}}_{\text{CBF}}$ n'est pas une entrée du calcul mais la quantité *dérivée* qui mesure l'intervention du filtre, définie comme l'écart entre la vitesse projetée et la vitesse soumise :

$$\Delta \dot{\mathbf{q}}_{\text{CBF}} \equiv \dot{\mathbf{q}}_{\text{proj}} - \dot{\mathbf{q}}_{\text{base}}.$$

C'est cette quantité, nulle dès que la contrainte est satisfaite (le filtre est alors transparent) et de norme égale à l'intensité de la correction, qui est tracée dans la décomposition des vitesses du Chapitre ???. Dans le cas d'une *unique* contrainte active, la projection sur le seul demi-espace admet une forme fermée qui en donne l'expression explicite :

$$\Delta \dot{\mathbf{q}}_{\text{CBF}} = \frac{\max(0, b(h) - \nabla h^\top \dot{\mathbf{q}}_{\text{in}})}{\|\nabla h\|^2 + \varepsilon} \nabla h.$$

Lorsque plusieurs contraintes agissent simultanément (Section ???), $\dot{\mathbf{q}}_{\text{proj}}$ résulte d'une projection itérative sur l'ensemble des demi-espaces et $\Delta \dot{\mathbf{q}}_{\text{CBF}}$ n'a plus d'expression close ; la correction y reste bornée en norme afin de limiter l'agressivité de l'intervention.

La vitesse projetée est ensuite saturée aux limites articulaires :

$$\dot{\mathbf{q}}_{\text{proj,sat}} = \text{sat}(\dot{\mathbf{q}}_{\text{proj}}).$$

Cette vitesse saturée est filtrée par un second filtre passe-bas exponentiel de constante de temps τ_{safe} :

$$\dot{\mathbf{q}}_{f,k} = (1 - \gamma_{\text{safe}}) \dot{\mathbf{q}}_{f,k-1} + \gamma_{\text{safe}} \dot{\mathbf{q}}_{\text{proj,sat},k},$$

avec

$$\gamma_{\text{safe}} = \frac{\Delta t}{\Delta t + \tau_{\text{safe}}}.$$

L'ordre saturation puis filtrage évite que le filtre mémorise une vitesse physiquement irréalisable.

La saturation et le filtrage peuvent toutefois réintroduire une légère violation de la contrainte CBF. La contrainte est donc réévaluée sur la vitesse filtrée :

$$c_f = \nabla h^\top \dot{\mathbf{q}}_f + \kappa_{\text{eff}} h - m_{\text{CBF}},$$

où

$$\kappa_{\text{eff}} = \begin{cases} \kappa_{\text{safe}}, & h \geq 0, \\ \kappa_{\text{rec}}, & h < 0. \end{cases}$$

Si $c_f < 0$ dans la bande d'activation, une réparation finale est appliquée :

$$\Delta \dot{\mathbf{q}}_{\text{rep}} = \frac{\max(0, -c_f)}{\|\nabla h\|^2 + \varepsilon} \nabla h.$$

Cette réparation est une projection corrective sur la même contrainte CBF linéarisée. Elle ne constitue pas une seconde évaluation complète du CBF, car elle réutilise la même valeur de

barrière h et le même gradient ∇h que la projection principale.

La vitesse finale envoyée à la commande est donc

$$\dot{\mathbf{q}}_{\text{final}} = \text{sat}(\dot{\mathbf{q}}_f + \Delta \dot{\mathbf{q}}_{\text{rep}}).$$

Le contrôleur bas niveau du robot étant un contrôleur de *vitesse* articulaire, cette vitesse certifiée constitue directement la commande envoyée à l'actionneur :

$$\dot{\mathbf{q}}_{\text{cmd},k} = \dot{\mathbf{q}}_{\text{final},k},$$

sans étape d'intégration en une consigne de position. La commande de sécurité reste ainsi exactement la vitesse que la contrainte CBF a certifiée au cycle courant : aucune dynamique d'intégration ne s'interpose entre la projection et l'actionneur, et la garantie en temps discret se réduit au seul maintien de cette vitesse pendant une période de commande (bloqueur d'ordre zéro), déjà comptabilisé dans la marge d_{safe} (Section ??). Une référence de position intégrée $\mathbf{q}_{\text{ref},k} = \mathbf{q}_{\text{ref},k-1} + \dot{\mathbf{q}}_{\text{final},k} \Delta t$ est néanmoins maintenue en parallèle, à des fins de supervision et de visualisation, mais elle n'est pas l'entrée du contrôleur. Lorsque la contrainte est inactive et que la projection ne modifie pas la vitesse nominale, $\dot{\mathbf{q}}_{\text{final}}$ se réduit à la vitesse nominale lissée, transmise telle quelle.

4.7.6 Extension multi-contraintes et portée des garanties

La projection sur un demi-espace unique traite un seul obstacle critique à la fois : deux obstacles menaçant simultanément des liens différents sont agrégés en une contrainte moyenne. La variante multi-contraintes attribue un demi-espace à chaque point actif. Les valeurs h_i de l'ensemble des points sélectionnés sont d'abord évaluées sans gradient, puis l'ensemble actif $\mathcal{A}$ des K_a points les plus critiques vérifiant $h_i \leq h_{\text{act}}$ est retenu, et la matrice des gradients $\mathbf{G} = [\nabla h_1, \dots, \nabla h_{K_a}]^\top$ est obtenue par un unique produit vecteur-jacobienne groupé. Le problème (??), étendu à un demi-espace par contrainte active,

$$\min_{\dot{\mathbf{q}}} \frac{1}{2} \|\dot{\mathbf{q}} - \dot{\mathbf{q}}_{\text{base}}\|^2 \quad \text{s.c.} \quad \nabla h_i^\top \dot{\mathbf{q}} \geq b(h_i) \quad \forall i \in \mathcal{A},$$

n'admet plus de solution en forme close ; il est résolu par projections successives relaxées (méthode de Gauss-Seidel sur les demi-espaces) :

$$\dot{\mathbf{q}} \leftarrow \dot{\mathbf{q}} + \omega \frac{\max(0, b(h_i) - \nabla h_i^\top \dot{\mathbf{q}})}{\|\nabla h_i\|^2 + \varepsilon} \nabla h_i,$$

itérées N_{it} fois sur l'ensemble actif. À cette dimension (7 variables, K_a contraintes), la résolution s'effectue sur CPU en une fraction de milliseconde après un unique transfert GPU–CPU des quantités $\mathbf{G}$, b et $\dot{\mathbf{q}}_{base}$.

Au-delà de cette extension, il convient de délimiter ce que le filtre garantit réellement. L'invariance avant théorique de l'ensemble $\mathcal{C}$ suppose une dynamique en temps continu, un SDF exact, des obstacles correctement observés et immobiles, et un suivi parfait de la commande par le contrôleur bas niveau. L'implémentation s'écarte de ces hypothèses sur quatre points quantifiables : la discrétisation de la commande (75 Hz), l'obsolescence de l'ensemble des points critiques (mise à jour à 30 Hz), l'erreur d'approximation du SDF de Bernstein (caractérisée au Chapitre ??) et le biais conservateur du soft-min ($\alpha \log N$). Le système constitue donc un filtrage réactif de sécurité dont la garantie pratique repose sur le dimensionnement de la marge d_{safe} pour absorber la somme de ces termes, validé expérimentalement au Chapitre ??.

Tableau 4.2 Paramètres du filtre de sécurité SDF-CBF.

Symbole	Description	Valeur
d_{safe}	marge de sécurité du SDF	15 mm
α	température du soft-min	0,001
K	points critiques sélectionnés	100
h_{act}	bande d'activation de la contrainte	6 mm
v_{in}	vitesse d'approche normale maximale	0,20 rad/s
v_{rec}	vitesse de sortie exigée en violation	0 rad/s
d_{rec}	profondeur de saturation en récupération	10 mm
Δv_{max}	correction de projection maximale	0,35 rad/s
m_{CBF}	marge numérique de la contrainte	0,001
$\kappa_{safe}, \kappa_{rec}$	gains de la réparation finale	3, 5
$K_p, \dot{q}_{fb,max}$	suivi proportionnel et saturation	5, 0,5 rad/s
τ_{nom}, τ_{safe}	constantes des filtres passe-bas	0,05 s, 0,2 s
Δq_{lead}	avance maximale de la référence	0,05 rad
K_a, N_{it}, ω	QP multi-contraintes	4, 3, 1,0

4.8 Intégration temps réel

4.8.1 Ordonnancement des modules

Explication pourquoi l'architecture ROS a cette allure.

Présentation des messages ROS (Publishers/Subscribers) pour générer des trajectoires robotiques et les exécuter en les contraignant avec le CBF

4.8.2 Justification des fréquences de contrôle

Inférence modèle Flow Matching / Temps de calcul de contrainte, de SAM2, etc.

Gestion des pertes de la cibles

CHAPITRE 5 RÉSULTATS ET DISCUSSION

Présentation avec CBF et sans CBF

Présentation de la détection avec SAM 3

Listes des figures pour les résultats

- Trajectoire de la pointe de la fourchette dans le nuage persistant map ;
 1. Nuage de points de la scène ;
 2. Trajectoire sans CBF
 3. Trajectoire avec CBF
 4. Cible.
- Évolution de $h(q)$ en fonction du temps ;
 1. h_{corr} ;
 2. ligne zéro.
 3. zones où le CBF est actif (en rouge par exemple).
- Distance obstacle-robot en fonction du temps
- Décomposition des vitesses en fonction du temps
 1. $|dq_{ff}|$;
 2. $|dq_{feedback}|$;
 3. $|dq_{base}|$
 4. $|dq_{cbf_delta}|$
 5. $|dq_{escape}|$
 6. $|dq_{safe}|$.
 7. Objectif : Expliquer le comportement du système et montré quand le FLOW Matching dirige le robot seul, quand le CBF intervient, quand l'échappement est nécessaire, si la vitesse corrigée devient trop agressive, etc.
- Effet du filtre passe bas
 1. Vitesse avant CBF brute
 2. Vitesse avant CBF filtrée
- Temps de calcul du pipeline. Faire un boxplot (ou un tableau) sur :
 1. Temps de détection avec SAM3

2. Temps de suivi avec SAM2
 3. Temps de mise à jour persistent map
 4. Temps de génération de trajectoire avec Flow Matching
 5. Temps de calcul de la cinématique inverse
 6. Temps de calcul des points critiques
 7. Temps de calcul de la correction de trajectoire
 8. Temps total du pipeline
- Résistance faces aux occlusions et pertes de détection.
 1. Détection dispo / indispo dans le temps
 2. Images présentant pertes de détection puis redétection
 - NE PAS OUBLIER LA DÉTECTION INITIALE DES FORK TEETH
 - Cas d'échec (comme la figure de Félix).

5.1 Protocol expérimental

5.1.1 Description des scénarios testés

Pick and place

Robot assisted feeding

5.1.2 Métriques utilisées

À compléter.

5.2 validation du modèle géométrique de sécurité

Cette section valide le modèle géométrique au cœur du filtre de sécurité : la **SDF!** (**SDF!**) du robot approximée par polynômes de Bernstein (section ??). Trois propriétés sont évaluées par rapport à la vérité terrain calculée sur les maillages du robot : la précision de la distance, la qualité de la direction du gradient et le temps de calcul.

Le protocole est le suivant : pour chacun des six corps protégés, des rayons sont lancés depuis la surface le long de la normale sortante (40 rayons par corps), et la **SDF!** est évaluée en 25 échantillons par rayon, entre 1 mm et 50 mm de la surface. Seuls les rayons *propres* sont conservés, c'est-à-dire ceux dont la distance exacte à l'union des corps coïncide avec l'avancée le long de la normale, ce qui garantit que l'axe des abscisses des figures correspond bien à la

distance réelle à la surface du robot. Les échantillons situés hors du domaine d’entraînement du modèle d’un corps (7,7 % des points, principalement au-delà de 35 mm) sont exclus, car le modèle y retourne une borne géométrique plutôt que la **SDF!** apprise. Au total, 5541 points extérieurs vérifiés sont retenus.

5.2.1 Précision du SDF du robot

La figure ?? compare la distance prédite à la distance exacte. L’erreur absolue moyenne est de 1,96 mm avec un biais négligeable ($-0,02$ mm) et une corrélation $R = 0,9869$. La distribution des erreurs signées est centrée et symétrique, avec un 99^e centile de l’erreur absolue de 6,8 mm.

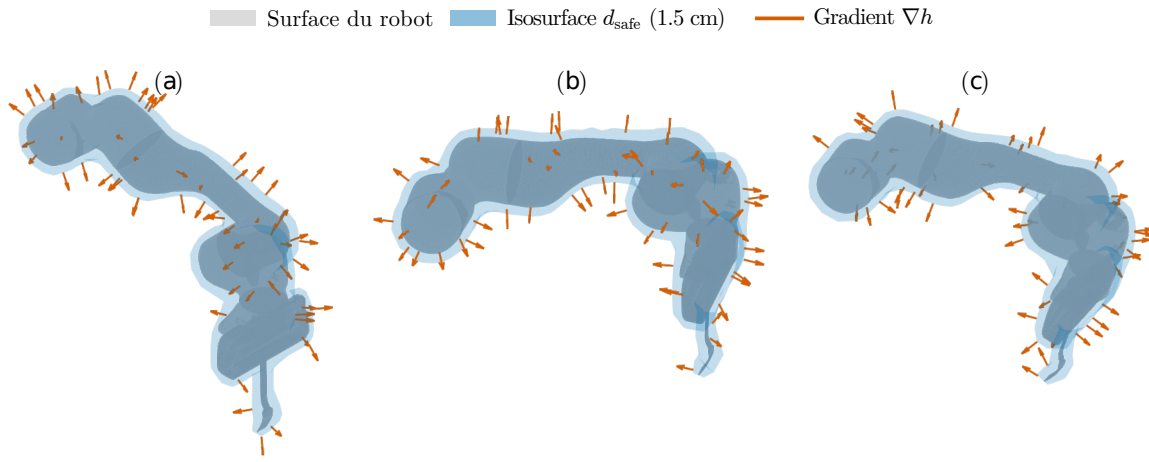


Figure 5.1 Précision de la distance du **SDF!** de Bernstein. (a) Profil de distance le long des normales à la surface du robot ; (b) distribution de l’erreur signée par rapport à la distance exacte calculée sur les maillages.

La figure ??(a) présente l’erreur angulaire entre le gradient du modèle et la direction exacte (vecteur vers le point de surface le plus proche). L’erreur médiane globale est de $5,4^\circ$, et reste sous 10° pour la grande majorité des points dès 5 mm de la surface. La figure ??(b) montre la norme du gradient : une **SDF!** exacte satisfait la propriété eikonale $\|\nabla h\| = 1$, et le modèle s’en approche au-delà de 10 mm, avec un affaissement vers 0,7 tout près de la surface, où le polynôme lisse le passage par zéro.

Ces résultats justifient la marge de sécurité $d_{\text{safe}} = 15$ mm retenue au chapitre précédent : l’erreur de distance au 99^e centile (6,8 mm) reste largement inférieure à la marge, de sorte qu’une sous-estimation de la distance par le modèle ne suffit pas à amener le robot au contact. De plus, à d_{safe} , la direction du gradient (erreur médiane $\approx 6^\circ$) et sa norme (≈ 1) sont déjà bien conditionnées, ce qui assure que la contrainte **CBF!** (**CBF!**) pousse le robot dans la

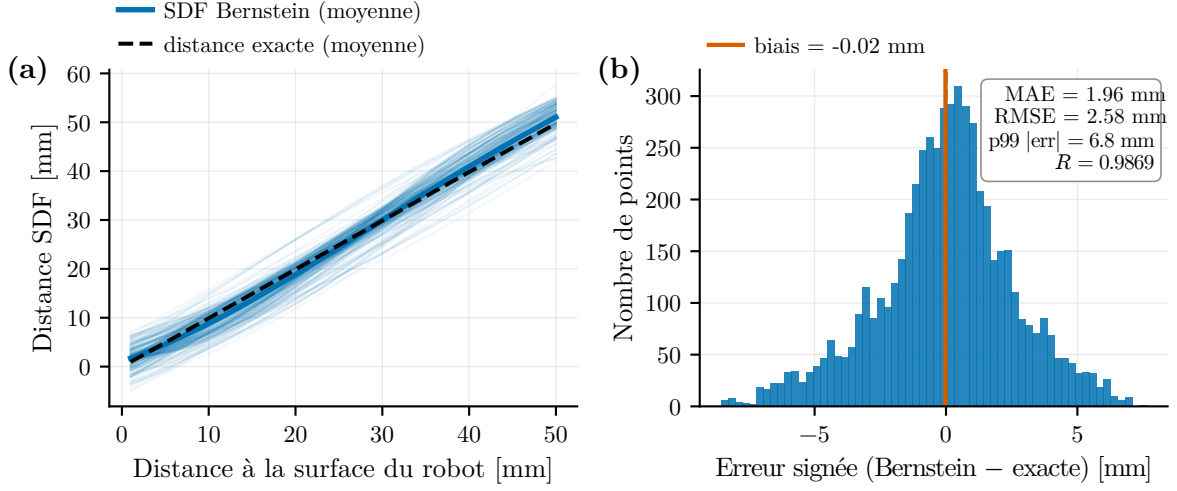


Figure 5.2 Qualité du gradient du **SDF!** en fonction de la distance à la surface du robot. (a) Erreur de direction par rapport au gradient exact; (b) norme du gradient (propriété eikonale).

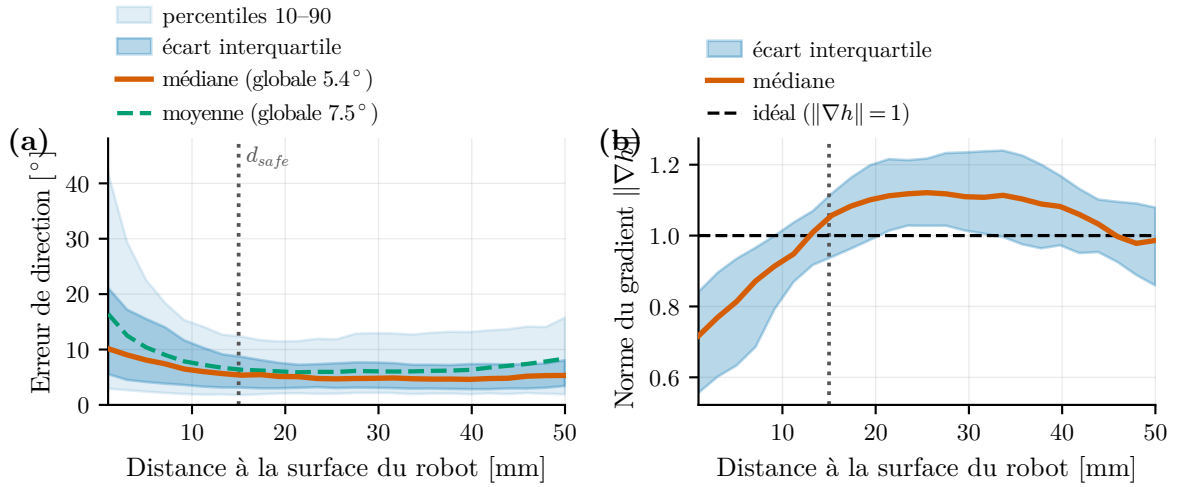


Figure 5.3 Qualité du gradient du **SDF!** en fonction de la distance à la surface du robot. (a) Erreur de direction par rapport au gradient exact; (b) norme du gradient (propriété eikonale).

bonne direction au moment où le filtre s'active.

5.2.2 Temps d'évaluation du SDF et du gradient

La figure ?? compare le temps d'évaluation conjointe de la distance et du gradient entre le modèle de Bernstein (analytique, sur **GPU!** (**GPU!**)) et la vérité terrain par requête de proximité sur les maillages (**trimesh**, sur **CPU!** (**CPU!**)). Le temps du modèle forme un plateau d'environ 5 ms jusqu'à quelques milliers de points : à ces tailles, le coût est dominé par des frais fixes par appel (cinématique directe, $\approx 2,5$ ms, lancement des noyaux **GPU!**, évaluation du gradient et transferts mémoire), le coût par point étant négligeable tant que le **GPU!** n'est pas saturé. La croissance ne devient visible qu'au-delà d'environ 4096 points, soit bien après le point d'opération du filtre ($K = 100$ points critiques). Au point d'opération, l'évaluation reste donc largement en deçà du budget de 13,3 ms imposé par la boucle de commande à 75 Hz, tandis que la requête sur maillage croît linéairement avec le nombre de points et dépasse ce budget d'un à deux ordres de grandeur, ce qui exclut son utilisation en ligne.

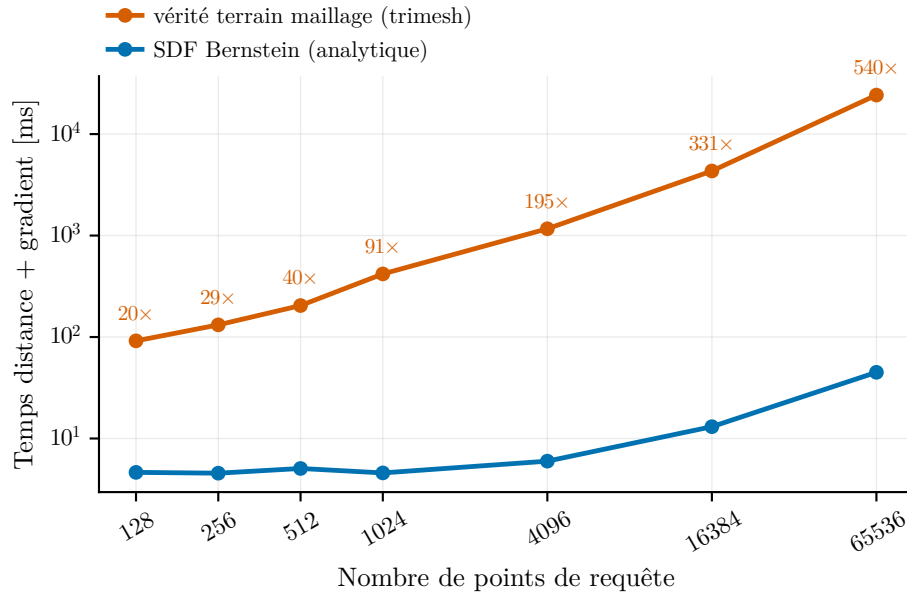


Figure 5.4 Temps de calcul de la distance et du gradient : modèle de Bernstein (**GPU!**) contre vérité terrain sur maillages (**CPU!**), en fonction du nombre de points de requête.

5.2.3 Visualisation des points critiques sélectionnés

Enveloppe de sécurité correspondant à l'isosurface ($d_{\text{robot}} = 1,5$ cm).

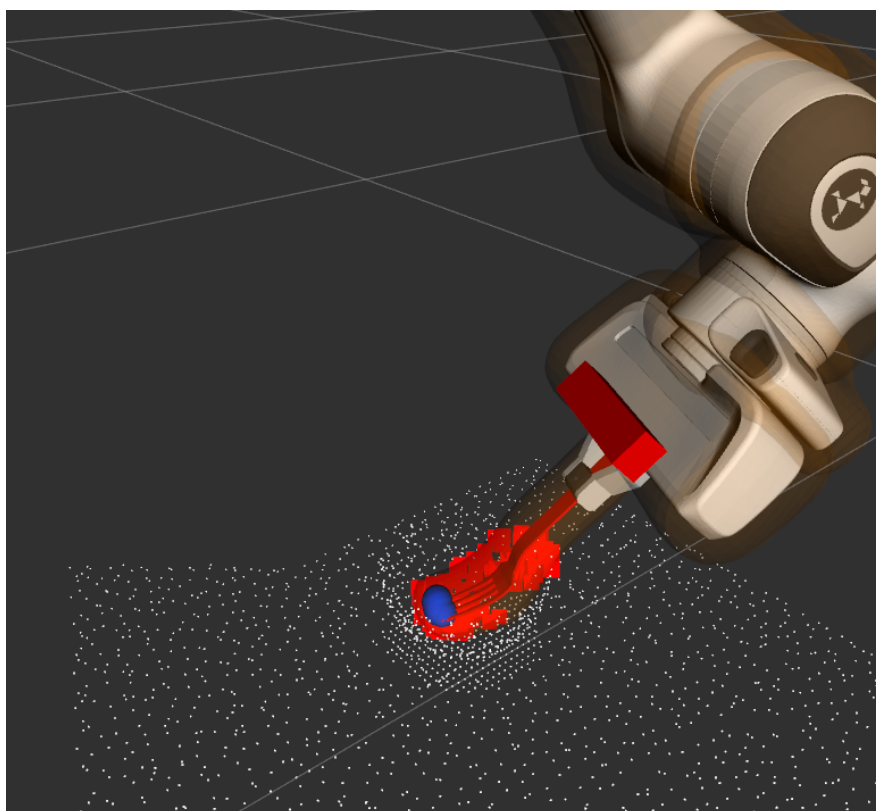


Figure 5.5 Sélection des points les plus proches pour contraindre le CBF

5.3 Qualité de la génération nominale

5.3.1 Trajectoire générée sans obstacle critique

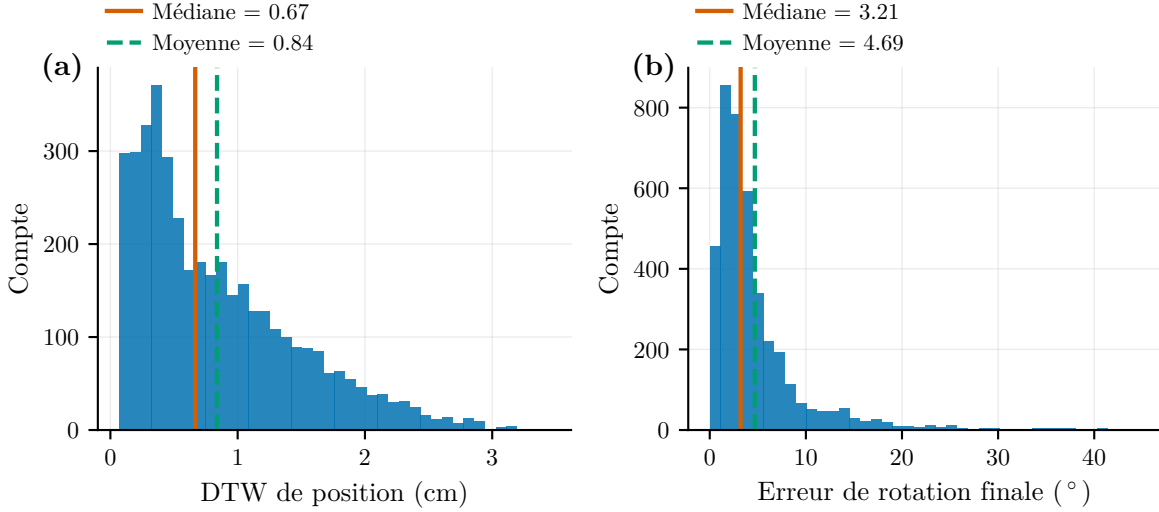


Figure 5.6 Qualité du gradient du **SDF!** en fonction de la distance à la surface du robot. (a) Erreur de direction par rapport au gradient exact; (b) norme du gradient (propriété eikonale).

5.3.2 Cohérence de la trajectoire avec la cible

À compléter.

À compléter.

À compléter.

À compléter.

5.4 Sécurité et évitement d'obstacle

5.4.1 Comparaison des trajectoires avec et sans SDF-CBF

5.4.2 Évolution de $h(q)$ et de la contrainte de sécurité

La Figure ?? présente l'évolution de la barrière h_{soft} et de sa valeur compensée h_{corr} au cours d'un essai représentatif. Les bandes vertes marquent les cycles où la projection corrige la vitesse ($\Delta \dot{\mathbf{q}}_{\text{CBF}} \neq 0$); les bandes jaunes, ceux où seule la réparation intervient — la vitesse

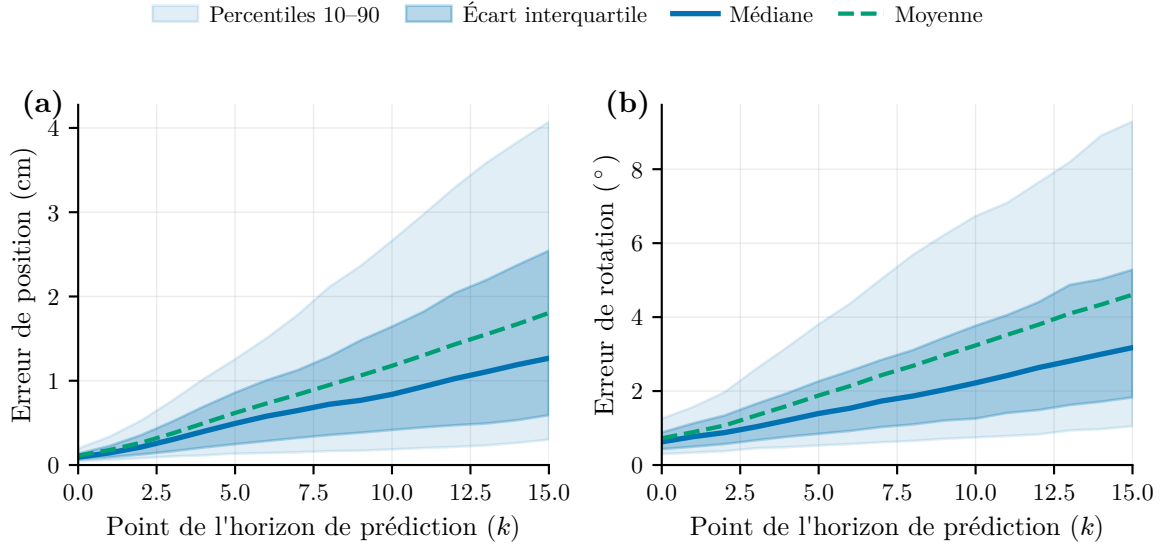


Figure 5.7 Qualité du gradient du **SDF!** en fonction de la distance à la surface du robot. (a) Erreur de direction par rapport au gradient exact; (b) norme du gradient (propriété eikonale).

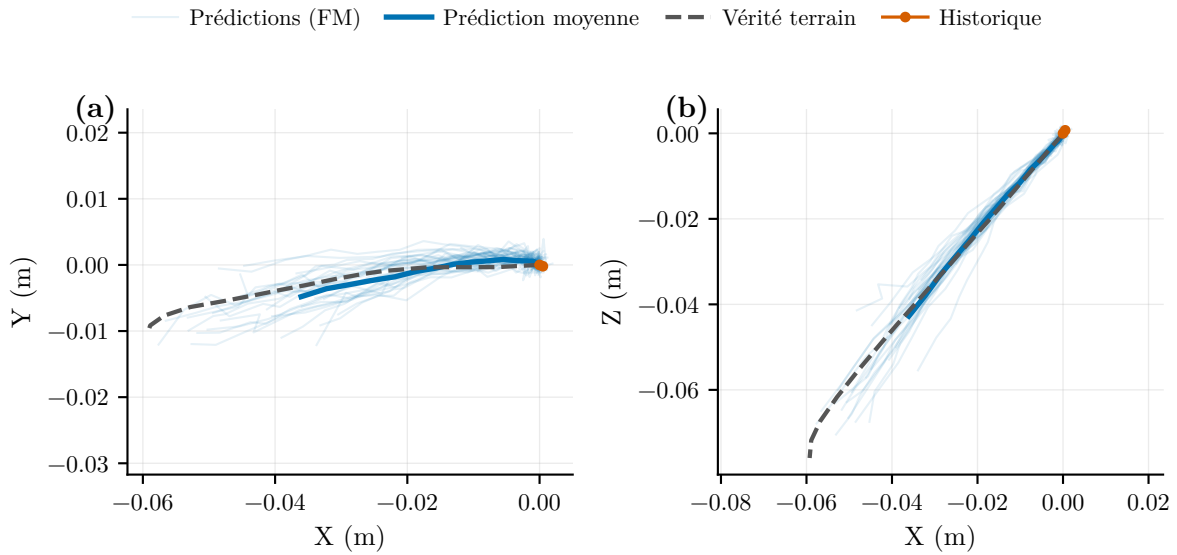


Figure 5.8 Qualité du gradient du **SDF!** en fonction de la distance à la surface du robot. (a) Erreur de direction par rapport au gradient exact; (b) norme du gradient (propriété eikonale).

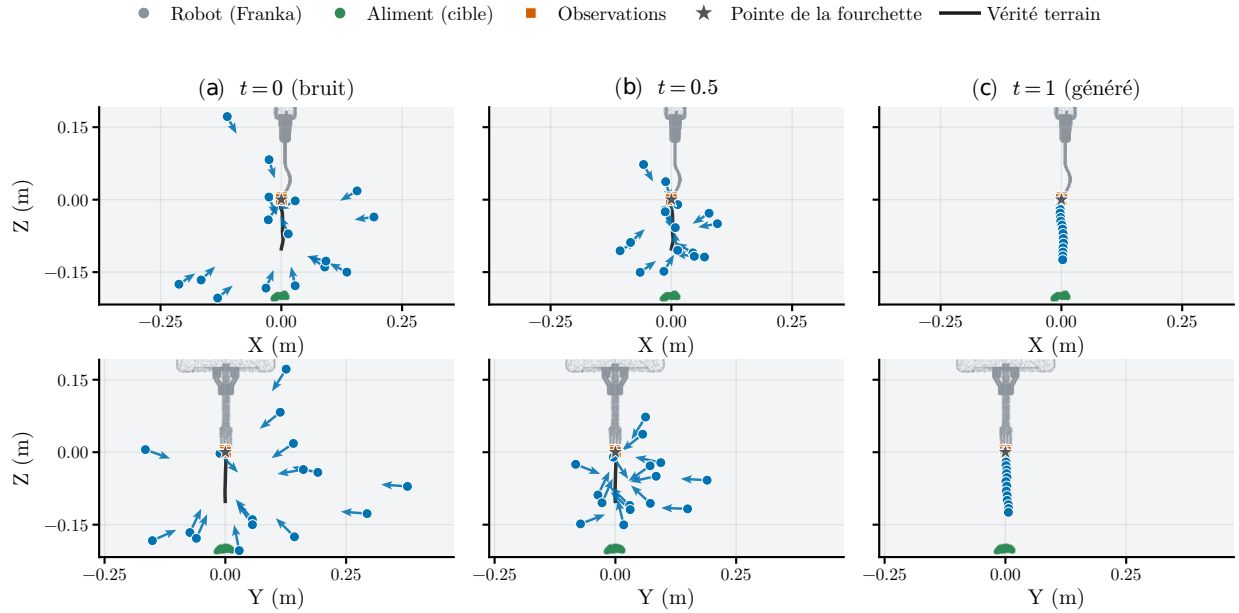


Figure 5.9 Qualité du gradient du **SDF!** en fonction de la distance à la surface du robot. (a) Erreur de direction par rapport au gradient exact; (b) norme du gradient (propriété eikonale).

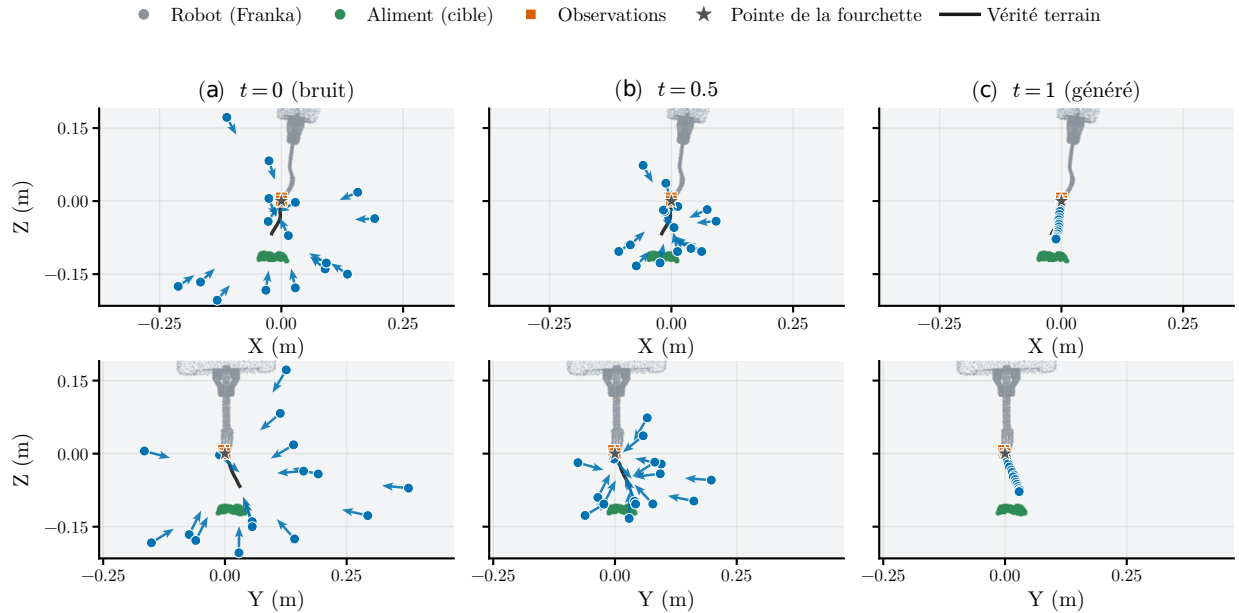


Figure 5.10 Qualité du gradient du **SDF!** en fonction de la distance à la surface du robot. (a) Erreur de direction par rapport au gradient exact; (b) norme du gradient (propriété eikonale).

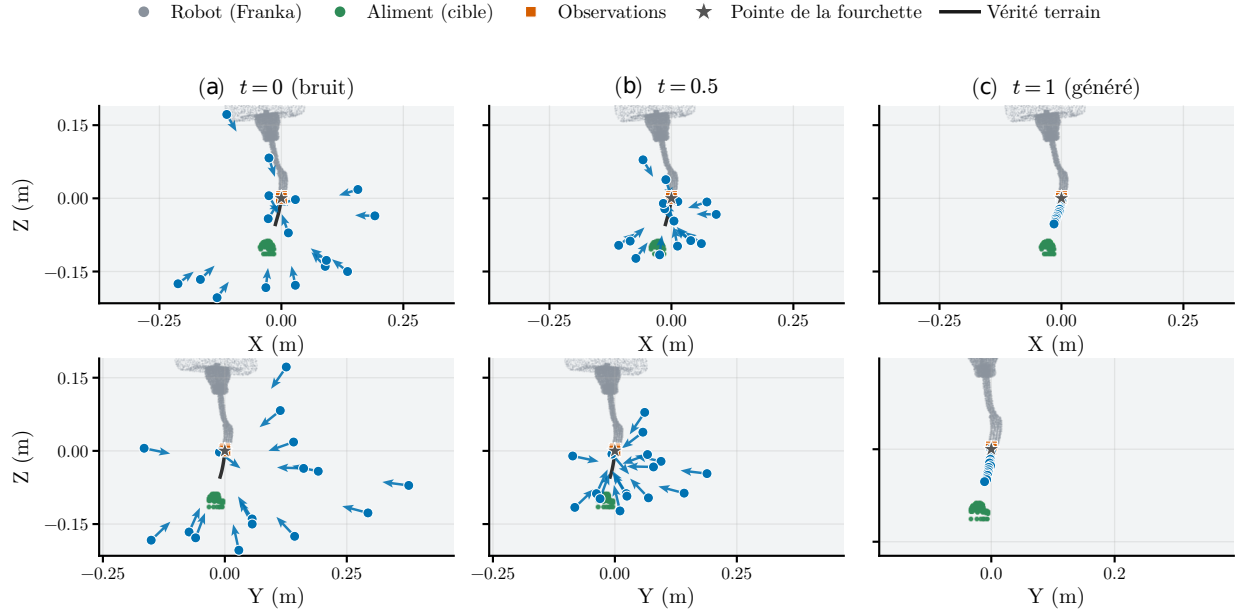


Figure 5.11 Qualité du gradient du **SDF**! en fonction de la distance à la surface du robot. (a) Erreur de direction par rapport au gradient exact; (b) norme du gradient (propriété eikonale).

instantanée satisfait la contrainte, mais la commande filtrée, qui porte la mémoire des cycles précédents, échoue à la vérification et est corrigée par $\Delta \dot{\mathbf{q}}_{\text{rep}}$. Après la phase d'approche, le système opère en régime de suivi de frontière : h_{corr} se stabilise à quelques millimètres de la frontière pendant que le robot progresse tangentiellement le long des surfaces vers la cible, conformément au comportement attendu de la projection préservant la composante tangentielle (Section ??).

La Figure ?? compare la valeur de la contrainte de sécurité évaluée sur la vitesse nominale (avant projection) et sur la commande finale (après filtrage et réparation). La courbe nominale plonge sous zéro dès que le robot opère à proximité des obstacles : laissée seule, la trajectoire générée par *Flow Matching* violerait continuellement la condition de sécurité. La courbe finale ne descend jamais sous $-m_{\text{CBF}}$ (pire valeur -5×10^{-4}) : le résidu introduit par le filtrage et les saturations est entièrement absorbé par la marge numérique, de sorte que la condition CBF proprement dite, $\nabla h^\top \dot{\mathbf{q}} + \kappa_{\text{eff}} h \geq 0$, est satisfaite à chaque cycle de l'essai. L'étape de réparation (Section ??) est loin d'être marginale dans ce résultat : elle intervient sur la quasi-totalité des cycles où la projection est active, le filtre de sortie érodant systématiquement une partie de la correction au sein du même cycle. L'écart entre les deux courbes constitue la mesure directe de l'intervention du filtre de sécurité.

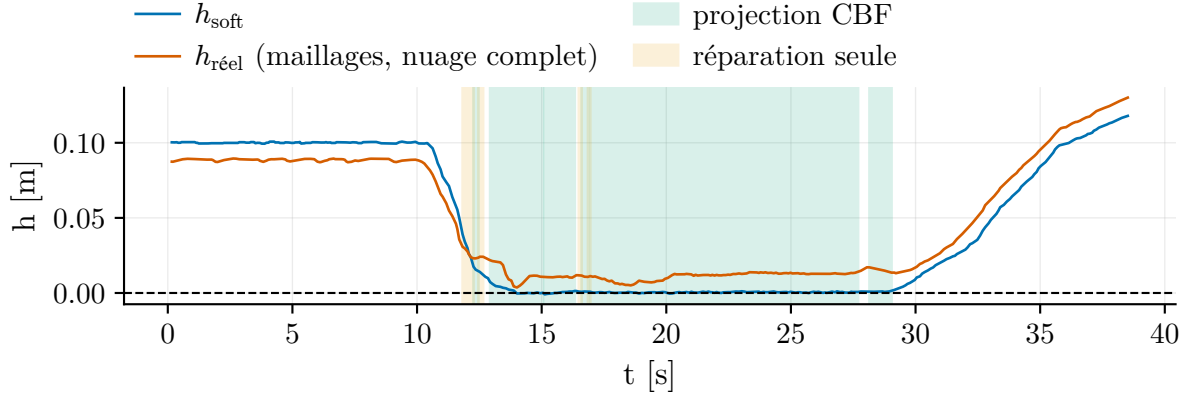


Figure 5.12 Évolution de la barrière h_{soft} et de sa valeur compensée h_{corr} pour une tâche en présence d'obstacles ; les bandes vertes indiquent les cycles où la projection corrige la vitesse, les bandes jaunes ceux où seule la réparation intervient.

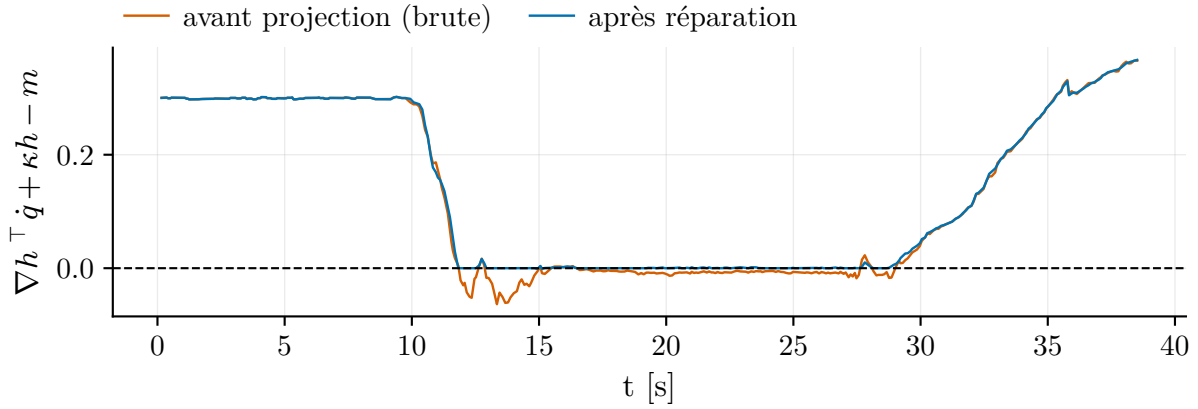


Figure 5.13 Contrainte de sécurité évaluée sur la vitesse nominale (avant projection) et sur la commande finale (après réparation). Les valeurs négatives de la courbe nominale correspondent aux violations qu'aurait commises la trajectoire générée sans filtre.

5.4.3 Déviation entre trajectoire nominale et trajectoire sécurisée

À compléter.

5.5 Analyse des vitesses et de la fluidité

5.5.1 Décomposition des vitesses articulaires

La Figure ?? déroule la chaîne complète des Sections ?? à ??. Le panneau (a) montre la construction de la vitesse nominale : le feedforward retimé $\dot{\mathbf{q}}_{\text{ff}}$, de norme constante par construction, la correction de suivi $\dot{\mathbf{q}}_{\text{fb}}$, et la vitesse nominale filtrée $\dot{\mathbf{q}}_{\text{base}}$ qui en résulte. Le panneau (b) montre ce que le filtre de sécurité en fait : les deux corrections, projection $\Delta\dot{\mathbf{q}}_{\text{CBF}}$ et réparation $\Delta\dot{\mathbf{q}}_{\text{rep}}$, et la vitesse effectivement commandée $\dot{\mathbf{q}}_{\text{final}}$, superposées à $\dot{\mathbf{q}}_{\text{base}}$ comme référence commune aux deux panneaux. La réparation n'étant pas journalisée par le nœud, $\Delta\dot{\mathbf{q}}_{\text{rep}}$ est reconstituée en rejouant le filtre passe-bas de sortie sur les signaux enregistrés, reconstruction validée par un résidu médian de 10^{-4} rad/s sur les cycles sans réparation. Les normes sont exprimées dans l'espace articulaire (rad/s) ; la vitesse cartésienne de l'outil s'en déduit par la jacobienne et varie avec la configuration du bras.

Trois observations ressortent. Premièrement, $\|\dot{\mathbf{q}}_{\text{ff}}\|$ est constante à 0,25 rad/s sur toute la tâche : le retiming à vitesse articulaire constante (Section ??) remplit son rôle et fournit au filtre une référence régulière. Deuxièmement, la correction de projection est strictement nulle hors des zones d'activation : le filtre est transparent dès que la contrainte est satisfaite, ce qui confirme le mécanisme de transmission directe. On remarque d'ailleurs des épisodes où $\Delta\dot{\mathbf{q}}_{\text{rep}}$ est active alors que $\Delta\dot{\mathbf{q}}_{\text{CBF}}$ demeure nulle (vers 5 s et 22 s) : la barrière y effleure la frontière sans la presser — la projection juge la vitesse instantanée admissible, mais la commande filtrée, qui porte la mémoire des cycles précédents, échoue à la revérification et est corrigée par la réparation. Troisièmement, la vitesse nominale $\dot{\mathbf{q}}_{\text{base}}$ présente un bruit important dont l'analyse spectrale situe la puissance dominante à 12 Hz, soit exactement la fréquence de régénération du planificateur : chaque nouvelle trajectoire générée déplace légèrement la consigne, et le gain de suivi K_p transforme ces sauts en impulsions de vitesse.

5.5.2 Action du filtre de sécurité sur la vitesse commandée

La Figure ?? superpose la vitesse nominale $\dot{\mathbf{q}}_{\text{base}}$ (ce que le robot ferait en l'absence d'obstacle) et la vitesse commandée $\dot{\mathbf{q}}_{\text{final}}$. Sur les cycles où le filtre intervient effectivement (projection ou réparation), la surface entre les deux courbes est teintée selon le sens de l'intervention : freinage lorsque le filtre retire de la vitesse, poussée lorsqu'il en ajoute (récupération hors de

la marge). L'écart entre les deux courbes se lit donc directement comme l'amplitude de la modification. Hors des zones d'activation, $\|\dot{\mathbf{q}}_{\text{final}}\|$ demeure légèrement sous $\|\dot{\mathbf{q}}_{\text{base}}\|$ ($\approx 5\%$ en moyenne) : ce n'est pas une intervention de sécurité mais l'effet du filtre passe-bas de sortie ($\tau_{\text{safe}} = 0,2$ s), qui, en moyennant des vecteurs dont la direction fluctue d'un cycle à l'autre, produit une norme inférieure à la moyenne des normes — le même mécanisme d'annulation vectorielle que celui décrit à la Section ??.

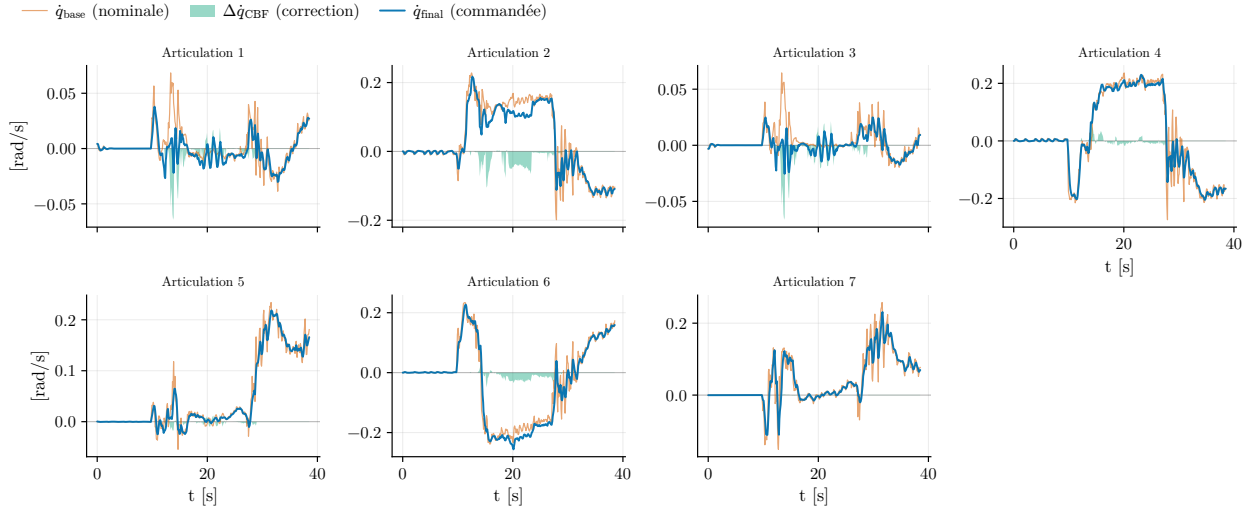


Figure 5.14 Action du filtre de sécurité : vitesse nominale et vitesse commandée superposées. Les surfaces teintées, restreintes aux cycles où le filtre intervient (projection ou réparation), distinguent le freinage de la poussée de récupération ; l'écart résiduel non teinté est l'effet du filtre passe-bas de sortie.

5.5.3 Effet du rééchantillonnage du planificateur sur la vitesse exécutée

La décomposition révèle un effet systémique du fonctionnement en horizon glissant. Bien que $\|\dot{\mathbf{q}}_{\text{ff}}\| = 0,25$ rad/s et que la correction de suivi soit en moyenne alignée avec le feed-forward (composante moyenne de $+0,03$ rad/s le long de $\dot{\mathbf{q}}_{\text{ff}}$), la vitesse nominale filtrée ne moyenne que $\approx 0,15$ rad/s. La cause n'est pas un freinage mais un phénomène vectoriel : à chaque régénération, la *direction* instantanée de $\dot{\mathbf{q}}_{\text{ff}}$ saute (rotations atteignant 80° entre cycles consécutifs au 95° centile), et le filtre passe-bas, en moyennant des vecteurs dont la direction alterne à 12 Hz, produit un vecteur de norme inférieure à la moyenne des normes. Près de 40 % de la vitesse demandée est ainsi perdue par annulation vectorielle, indépendamment de tout obstacle. Ce constat motive l'agrégation temporelle des trajectoires générées (moyennage pondéré des dernières générations) comme amélioration future du planificateur.

5.5.4 Effet du filtrage sur la commande sécurisée

La Figure ?? compare la norme de la vitesse sécurisée avant et après le filtre passe-bas de sortie. Le filtre supprime la gigue de direction du gradient induite par le rafraîchissement de la sélection des points critiques à 30 Hz, au prix d'un retard de l'ordre de τ_{safe} ; la contrainte de sécurité est réévaluée après filtrage par l'étape de réparation (Section ??).

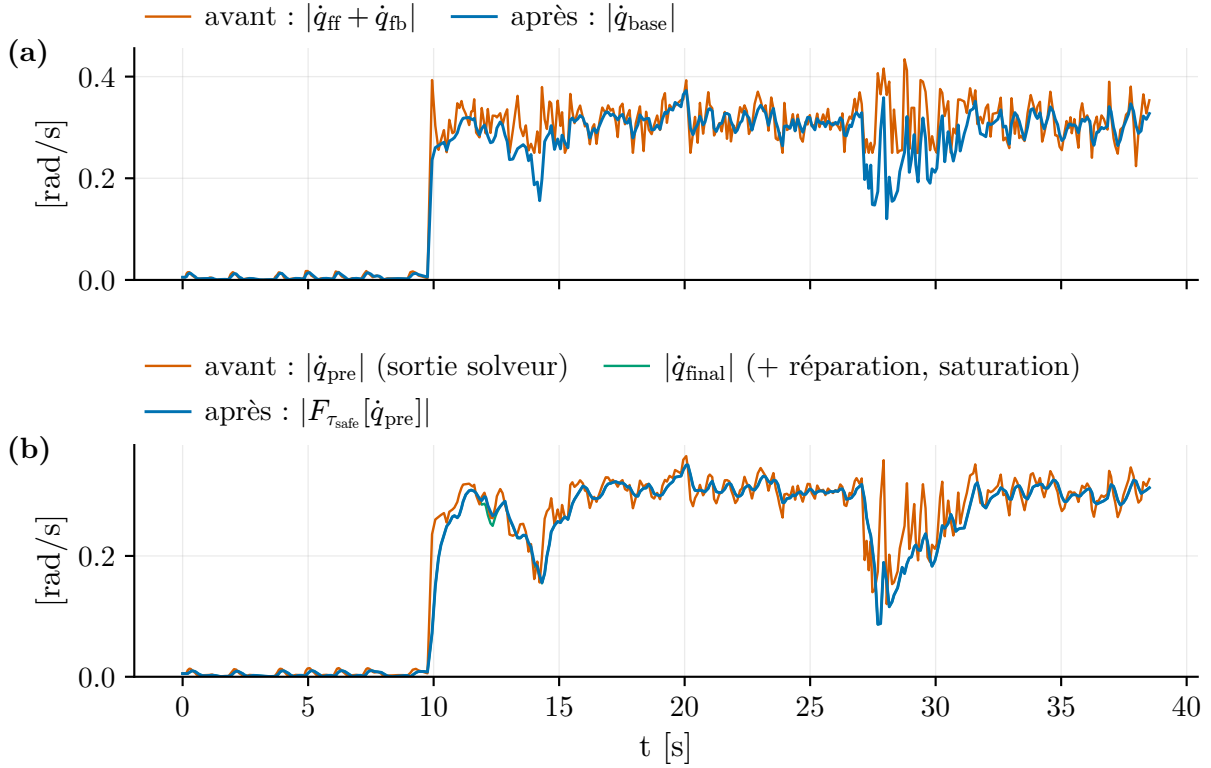


Figure 5.15 Norme de la vitesse sécurisée avant et après le filtre passe-bas de sortie ($\tau_{\text{safe}} = 0,2$ s).

5.5.5 Analyse de l'interface de commande

L'instrumentation du rapport entre le taux de variation de la barrière commandé ($\nabla h^\top \dot{q}_{\text{final}}$) et celui réellement exécuté ($\nabla h^\top \dot{q}_{\text{mes}}$) a mis en évidence deux modes de défaillance de l'interface entre le filtre de sécurité et le contrôleur de position bas niveau, justifiant a posteriori les choix de conception de la Section ?. Avec une consigne ré-ancrée à chaque cycle sur la position mesurée, le rapport exécuté/commandé mesuré était proche de zéro : les corrections de sécurité étaient réabsorbées avant d'être réalisées, et le robot demeurerait immobilisé contre la frontière malgré une commande de dégagement soutenue. À l'inverse, avec un pas

d'intégration supérieur à la période réelle de la boucle, la référence avançait plus vite que la vitesse certifiée et le rapport devenait négatif et supérieur à l'unité en valeur absolue : le robot traversait la frontière en oscillant, produisant des violations répétées de la marge. L'interface retenue — intégration de la référence persistante au pas réel Δt avec borne d'avance — élimine ces deux régimes.

5.6 Performance et temps de calcul

Tableau 5.1 Statistiques descriptives par étape (stage)

Stage	n	Mean	std/min	Med	p95	Max
cbf_command	1660	1.88	2.12	1.38	4.98	31.10
cbf_correction	1660	7.58	5.41	6.50	17.65	41.44
cbf_total	1660	9.91	6.43	8.60	22.06	57.81
critical_point_selection	832	42.27	18.35	39.62	75.49	148.81
fm_encode	263	1.51	2.21	1.02	2.85	20.87
fm_generation	263	62.07	6.40	61.62	73.89	83.89
ik	255	0.27	0.09	0.25	0.43	0.93
persistent_evict	709	3.17	1.74	2.76	5.90	22.08
persistent_integrate	1753	1.02	1.06	0.84	1.80	23.85
sam2_track	132	53.85	15.10	52.77	78.66	91.14
sam3_detect	2	182.45	152.72	182.45	319.89	335.16

À compléter.

5.7 Performances globales et limites

5.7.1 Taux de succès des scénarios

Pick and place

Robot assisted feeding

5.7.2 Cas d'échec

À compléter.

5.8 Discussion

CHAPITRE 6 CONCLUSION

Texte / Text.

6.1 Synthèse des travaux / Summary of Works

Texte / Text.

6.2 Limitations de la solution proposée / Limitations

6.3 Améliorations futures / Future Research

Texte / Text.

RÉFÉRENCES

- [1] S. Karaman et E. Frazzoli, “Sampling-Based Algorithms for Optimal Motion Planning,” *The International Journal of Robotics Research (IJRR)*, vol. 30, n°. 7, p. 846–894, 2011.
- [2] I. A. Şucan, M. Moll et L. E. Kavraki, “The Open Motion Planning Library,” *IEEE Robotics & Automation Magazine*, vol. 19, n°. 4, p. 72–82, 2012.
- [3] S. Liu et P. Liu, “Robot Motion Planning Benchmarking and Optimization using Motion Planning Pipeline,” 2021, arXiv preprint.
- [4] M. Zucker *et al.*, “CHOMP : Covariant Hamiltonian Optimization for Motion Planning,” *The International Journal of Robotics Research (IJRR)*, vol. 32, n°. 9-10, p. 1164–1193, 2013.
- [5] J. Schulman *et al.*, “Motion Planning with Sequential Convex Optimization and Convex Collision Checking,” *The International Journal of Robotics Research (IJRR)*, vol. 33, n°. 9, p. 1251–1270, 2014.
- [6] B. Liu *et al.*, “Collision-free Motion Generation Based on Stochastic Optimization and Composite Signed Distance Field Networks of Articulated Robot,” 2023, arXiv preprint.
- [7] S. Tian, M. Zheng et X. Liang, “Warm-Starting Optimization-Based Motion Planning for Robotic Manipulators via Point Cloud-Conditioned Flow Matching,” 2025, arXiv preprint.
- [8] A. Haffemayer *et al.*, “Warm-Starting Collision-Free Model Predictive Control With Object-Centric Diffusion,” 2026, arXiv preprint.
- [9] B. D. Argall *et al.*, “A survey of robot learning from demonstration,” 2009, arXiv preprint.
- [10] T. Z. Zhao *et al.*, “Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware,” 2023, arXiv preprint.
- [11] P. Sundaresan, S. Belkhale et D. Sadigh, “Learning Visuo-Haptic Skewering Strategies for Robot-Assisted Feeding,” 2022, arXiv preprint.
- [12] R. Liu, A. Bhaskar et P. Tokekar, “Adaptive Visual Imitation Learning for Robotic Assisted Feeding Across Varied Bowl Configurations and Food Types,” 2024, arXiv preprint.
- [13] P. Florence *et al.*, “Implicit Behavioral Cloning,” 2021, arXiv preprint.
- [14] D. Jarrett, I. Bica et M. v. d. Schaar, “Strictly Batch Imitation Learning by Energy-based Distribution Matching,” 2021, arXiv preprint.

- [15] P. Florence, L. Manuelli et R. Tedrake, “Self-Supervised Correspondence in Visuomotor Policy Learning,” 2019, arXiv preprint.
- [16] J. Luo *et al.*, “Precise and Dexterous Robotic Manipulation via Human-in-the-Loop Reinforcement Learning,” 2025, arXiv preprint.
- [17] A. Gupta *et al.*, “Relay Policy Learning : Solving Long-Horizon Tasks via Imitation and Reinforcement Learning,” 2019, arXiv preprint.
- [18] S. M. Khansari-Zadeh et A. Billard, “Learning Stable Nonlinear Dynamical Systems With Gaussian Mixture Models,” 2011, arXiv preprint.
- [19] R. Wolf *et al.*, “Diffusion models for robotic manipulation : a survey,” XXXX, arXiv preprint.
- [20] J. Cho, M. Kim et S. Jung, “A Survey on Flow Matching for Robotic Trajectory Generation and Control,” 2025, arXiv preprint.
- [21] C. Chi *et al.*, “Visuomotor Policy Learning via Action Diffusion,” XXXX, arXiv preprint.
- [22] J. Ho, A. Jain et P. Abbeel, “Denoising Diffusion Probabilistic Models,” 2020, arXiv preprint.
- [23] M. Janner *et al.*, “Planning with Diffusion for Flexible Behavior Synthesis,” 2022, arXiv preprint.
- [24] S. Huang *et al.*, “Diffusion-based Generation, Optimization, and Planning in 3D Scenes,” 2023, arXiv preprint.
- [25] Y. Ze *et al.*, “3D Diffusion Policy : Generalizable Visuomotor Policy Learning via Simple 3D Representations,” 2024, arXiv preprint.
- [26] A. Prasad *et al.*, “Consistency Policy : Accelerated Visuomotor Policies via Consistency Distillation,” 2024, arXiv preprint.
- [27] Z. Wang *et al.*, “One-Step Diffusion Policy : Fast Visuomotor Policies via Diffusion Distillation,” 2024, arXiv preprint.
- [28] G. Lu *et al.*, “ManiCM : Real-time 3D Diffusion Policy via Consistency Model for Robotic Manipulation,” 2025, arXiv preprint.
- [29] Y. Lipman *et al.*, “Flow Matching for Generative Modeling,” dans *International Conference on Learning Representations (ICLR)*, 2023.
- [30] X. Liu, C. Gong et Q. Liu, “Flow Straight and Fast : Learning to Generate and Transfer Data with Rectified Flow,” dans *International Conference on Learning Representations (ICLR)*, 2023.
- [31] A. Tong *et al.*, “Improving and Generalizing Flow-Based Generative Models with Mini-batch Optimal Transport,” *Transactions on Machine Learning Research (TMLR)*, 2024.

- [32] E. Chisari *et al.*, “Learning Robotic Manipulation Policies from Point Clouds with Conditional Flow Matching,” 2024, arXiv preprint.
- [33] G. Yan *et al.*, “ManiFlow : A General Robot Manipulation Policy via Consistency Flow Training,” 2025, arXiv preprint.
- [34] Z. Geng *et al.*, “Mean Flows for One-step Generative Modeling,” 2025, arXiv preprint.
- [35] H. Fang *et al.*, “OMP : One-step Meanflow Policy with Directional Alignment,” 2026, arXiv preprint.
- [36] S. Jiang *et al.*, “Streaming Flow Policy : Simplifying diffusion/flow-matching policies by treating action trajectories as flow trajectories,” 2025, arXiv preprint.
- [37] K. Nguyen *et al.*, “FlowMP : Learning Motion Fields for Robot Planning with Conditional Flow Matching,” 2025, arXiv preprint.
- [38] D. Soleymanzadeh, X. Liang et M. Zheng, “Flow Motion Policy : Manipulator Motion Planning with Flow Matching Models,” 2026, arXiv preprint.
- [39] M. Braun *et al.*, “Riemannian Flow Matching Policy for Robot Motion Learning,” 2024, arXiv preprint.
- [40] A. Brohan *et al.*, “RT-2 : Vision-Language-Action Models Transfer Web Knowledge to Robotic Control,” 2023, arXiv preprint.
- [41] M. J. Kim *et al.*, “OpenVLA : An Open-Source Vision-Language-Action Model,” 2024, arXiv preprint.
- [42] R. Sapkota *et al.*, “Vision-Language-Action (VLA) Models : Concepts, Progress, Applications and Challenges,” 2026, arXiv preprint.
- [43] V. Roponen, “Foundation models in robotic manipulation,” XXXX, arXiv preprint.
- [44] J. Wen *et al.*, “TinyVLA : Towards Fast, Data-Efficient Vision-Language-Action Models for Robotic Manipulation,” 2025, arXiv preprint.
- [45] S. Liu *et al.*, “Vision-language model-driven scene understanding and robotic object manipulation,” 2024, arXiv preprint.
- [46] O. Mees, L. Hermann et W. Burgard, “What Matters in Language Conditioned Robotic Imitation Learning over Unstructured Data,” 2022, arXiv preprint.
- [47] N. Ingelhart *et al.*, “A Robotic Skill Learning System Built Upon Diffusion Policies and Foundation Models,” 2024, arXiv preprint.
- [48] G. Yin *et al.*, “CodeDiffuser : Attention-Enhanced Diffusion Policy via VLM-Generated Code for Instruction Ambiguity,” 2025, arXiv preprint.
- [49] Meta AI Research, “SAM 3 : Segment Anything with Concepts,” *arXiv preprint*, 2025.

- [50] A. Kirillov *et al.*, “Segment Anything,” dans *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023, p. 4015–4026.
- [51] N. Ravi *et al.*, “SAM 2 : Segment Anything in Images and Videos,” *arXiv preprint arXiv :2408.00714*, 2024.
- [52] D. Seita *et al.*, “ToolFlowNet : Robotic Manipulation with Tools via Predicting Tool Flow from Point Clouds,” 2022, arXiv preprint.
- [53] R. Yang *et al.*, “FP3 : A 3D Foundation Policy for Robotic Manipulation,” 2025, arXiv preprint.
- [54] H. Li *et al.*, “Language-Guided Object-Centric Diffusion Policy for Generalizable and Collision-Aware Manipulation,” 2025, arXiv preprint.
- [55] Y. Fu *et al.*, “CordViP : Correspondence-based Visuomotor Policy for Dexterous Manipulation in Real-World,” 2025, arXiv preprint.
- [56] X. Sun, Y. Chen et D. Rakita, “PRISM-DP : Spatial Pose-based Observations for Diffusion-Policies via Segmentation, Mesh Generation, and Pose Tracking,” 2025, arXiv preprint.
- [57] R. Yu *et al.*, “No Need for Real 3D : Fusing 2D Vision with Pseudo 3D Representations for Robotic Manipulation Learning,” 2025, arXiv preprint.
- [58] Z. Ni *et al.*, “VO-DP : Semantic-Geometric Adaptive Diffusion Policy for Vision-Only Robotic Manipulation,” 2025, arXiv preprint.
- [59] J. Jones *et al.*, “Beyond Sight : Finetuning Generalist Robot Policies with Heterogeneous Sensors via Language Grounding,” 2025, arXiv preprint.
- [60] J. Morris *et al.*, “DynoSAM : Open-Source Smoothing and Mapping Framework for Dynamic SLAM,” 2025, arXiv preprint.
- [61] Y. Jin *et al.*, “SE(3)-PoseFlow : Estimating 6D Pose Distributions for Uncertainty-Aware Robotic Manipulation,” 2025, arXiv preprint.
- [62] A. Hornung *et al.*, “OctoMap : An Efficient Probabilistic 3D Mapping Framework Based on Octrees,” *Autonomous Robots*, vol. 34, n^o. 3, p. 189–206, 2013.
- [63] H. Oleynikova *et al.*, “Voxblox : Incremental 3D Euclidean Signed Distance Fields for On-Board MAV Planning,” dans *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017, p. 1366–1373.
- [64] A. Millane *et al.*, “nvvblox : GPU-Accelerated Incremental Signed Distance Field Mapping,” dans *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- [65] W. B. Bedada et G. Palli, “Real Time Collision Avoidance with GPU-Computed Distance Maps,” 2024, arXiv preprint.

- [66] B. Sundaralingam *et al.*, “CuRobo : Parallelized Collision-Free Robot Motion Generation,” dans *IEEE International Conference on Robotics and Automation (ICRA)*, 2023, p. 8112–8119.
- [67] J. Lee *et al.*, “Learning Fast, Tool aware Collision Avoidance for Collaborative Robots,” 2025, arXiv preprint.
- [68] Y. Li *et al.*, “Representing robot geometry as distance fields : Applications to whole-body manipulation,” dans *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2024, p. 15 351–15 357.
- [69] A. Govoni *et al.*, “Real-time collision avoidance with robot distance fields in a task-priority framework,” *Robotics and Autonomous Systems*, vol. 200, p. 105394, 2026.
- [70] O. Khatib, “Real-Time Obstacle Avoidance for Manipulators and Mobile Robots,” *The International Journal of Robotics Research (IJRR)*, vol. 5, n^o. 1, p. 90–98, 1986.
- [71] H. Ding *et al.*, “Towards Safe Imitation Learning via Potential Field-Guided Flow Matching,” 2025, arXiv preprint.
- [72] A. D. Ames *et al.*, “Control Barrier Functions : Theory and Applications,” 2019, arXiv preprint.
- [73] P. Wieland et F. Allgöwer, “CONSTRUCTIVE SAFETY USING CONTROL BARRIER FUNCTIONS,” 2007, arXiv preprint.
- [74] A. D. Ames *et al.*, “Control Barrier Function Based Quadratic Programs for Safety Critical Systems,” 2017, arXiv preprint.
- [75] W. S. Cortez *et al.*, “Control Barrier Functions for Mechanical Systems : Theory and Application to Robotic Grasping,” 2019, arXiv preprint.
- [76] A. Agrawal et K. Sreenath, “Discrete Control Barrier Functions for Safety-Critical Control of Discrete Systems with Application to Bipedal Robot Navigation,” dans *Robotics : Science and Systems (RSS)*, 2017.
- [77] L. E. Beaver, “Optimal Control Barrier Functions : Maximizing the Action Space Subject to Control Bounds,” 2024, arXiv preprint.
- [78] J. Yang, S. Jang et S. Han, “SafeFlowMatcher : Safe and Fast Planning using Flow Matching with Control Barrier Functions,” 2026, arXiv preprint.
- [79] X. Dai *et al.*, “SafeFlow : Safe Robot Motion Planning with Flow Matching via Control Barrier Functions,” 2025, arXiv preprint.
- [80] J. Long *et al.*, “Constraining Streaming Flow Models for Adapting Learned Robot Trajectory Distributions,” 2026, arXiv preprint.

- [81] Y. Song *et al.*, “OmniGuide : Universal Guidance Fields for Enhancing Generalist Robot Policies,” 2026, arXiv preprint.
- [82] K. Mizuta et K. Leung, “CoBL-Diffusion : Diffusion-Based Conditional Robot Planning in Dynamic Environments Using Control Barrier and Lyapunov Functions,” 2024, arXiv preprint.
- [83] Z. Yang *et al.*, “UniConFlow : A Unified Constrained Flow-Matching Framework for Certified Motion Planning,” 2026, arXiv preprint.
- [84] T.-Y. Huang *et al.*, “SAD-Flower : Flow Matching for Safe, Admissible, and Dynamically Consistent Planning,” 2026, arXiv preprint.
- [85] K. Mizuta et K. Leung, “Unified Generation-Refinement Planning : Bridging Guided Flow Matching and Sampling-Based MPC for Social Navigation,” 2026, arXiv preprint.
- [86] R. Römer *et al.*, “From Demonstrations to Safe Deployment : Path-Consistent Safety Filtering for Diffusion Policies,” 2026, arXiv preprint.
- [87] E. Welte *et al.*, “FlowCorrect : Efficient Interactive Correction of Generative Flow Policies for Robotic Manipulation,” 2026, arXiv preprint.
- [88] A. Liao *et al.*, “Delay-Aware Diffusion Policy : Bridging the Observation-Execution Gap in Dynamic Tasks,” 2025, arXiv preprint.
- [89] X. Ye *et al.*, “RA-DP : Rapid Adaptive Diffusion Policy for Training-Free High-frequency Robotics Replanning,” 2025, arXiv preprint.
- [90] F. Xiang *et al.*, “SAPIEN : A simulated part-based interactive environment,” dans *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2020.

ANNEXE A DÉMO

Texte de l'annexe A. Remarquez que la phrase précédente se termine par une lettre majuscule suivie d'un point. On indique explicitement cette situation à $\text{\LaTeX}$ afin que ce dernier ajuste correctement l'espacement entre le point final de la phrase et le début de la phrase suivante.

ANNEXE B ENCORE UNE ANNEXE / ANOTHER APPENDIX

Texte de l'annexe B en mode «landscape».

ANNEXE C UNE DERNIÈRE ANNEXE / THE LAST APPENDIX

Texte de l'annexe C.